

Prefetch Unit (PFU) Reimplementation Design Spec

本文定义 Cortex-M7 `cm7pfu` 的行为兼容重实现规格。目标读者应能仅依据本文实现新的 PFU，并通过接口级验证复现原 Register Transfer Level (RTL，寄存器传输级) 设计的取指、预测、缓冲、异常和取消行为。本文不要求复制原 RTL 的门级结构、寄存器命名或物理存储阵列实现。

1. Overview And Architecture

1.1 目标、范围与术语

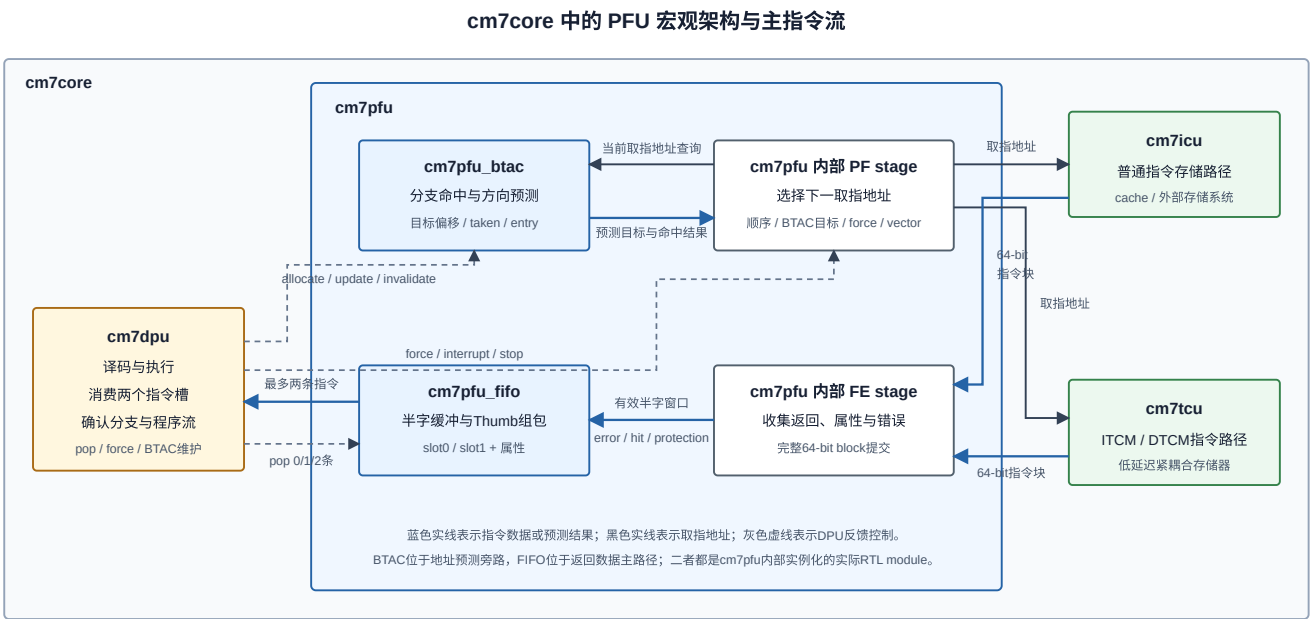
Prefetch Unit (PFU，预取单元) 位于处理器前端，负责产生指令地址、从 Instruction Cache Unit (ICU，指令缓存单元) 或 Tightly Coupled Memory Unit (TCU，紧耦合存储器单元) 取回指令、预测分支、把 Thumb 半字组成指令，并向 Data Processing Unit (DPU，数据处理单元) 最多提供两条指令。

本文中的主要术语如下。

术语	本文含义
Branch Target Address Cache (BTAC，分支目标地址缓存)	保存分支地址、目标偏移和方向历史，用于在分支执行前预测下一取指地址。
First-In First-Out (FIFO，先进先出队列)	保持指令到达顺序的缓冲结构；先写入的有效内容必须先被消费。
Tightly Coupled Memory (TCM，紧耦合存储器)	与 core 紧密连接、延迟可预测的存储区域；本 PFU 可从 ITCM 或 DTCM 取指。
Memory Protection Unit (MPU，内存保护单元)	按地址和 privilege 检查取指权限并返回 memory attributes。
Error Correction Code (ECC，错误纠正码)	用冗余校验位检测或纠正存储错误；本接口上的 ICU ECC error 触发 replay。
Main Stack Pointer (MSP，主堆栈指针)	reset vector table entry0 给出的初始主堆栈地址。
Memory Built-In Self-Test (MBIST，存储器内建自测)	启动或测试期间检查内部存储阵列的模式；有效时 PFU 保持 reset。
Reset All Registers (RAR，复位所有寄存器)	配置为 1 时，连非功能关键数据寄存器也接入 reset；为 0 时只保证 valid/control 状态复位。
Program Counter (PC，程序计数器)	指向当前或下一条架构指令的地址；vector PC 和 force PC 都会改变取指流。
Central Processing Unit (CPU，中央处理器)	本文中指 Cortex-M7 core； <code>cpuwait_i</code> 使其在 reset 后暂不开始取指。
Non-Maskable Interrupt (NMI，不可屏蔽中断)	不能被普通中断屏蔽的高优先级异常；其 vector fault 可进入 lockup 语义。

术语	本文含义
Interrupt Service Routine (ISR, 中断服务程序)	异常向量指向的处理程序；NVIC 提供的 ISR number 用于索引 vector table。
Negative, Zero, Carry, Overflow (NZCV, 负数/零/进位/溢出标志)	ARM 条件执行使用的四个状态标志；其中 Zero flag (Z, 零标志) 为 1 表示最近一次更新标志的运算结果为零。
P0 / P1	MPU 接口的相邻流水相位：P0 发送 lookup, P1 返回 abort 和 attributes。
De / Iss / Ex1 / Ex2 / Wr	DPU 的 decode、issue、两个 execute phase 和 write/retire phase；本文保留 RTL 的 stage 缩写。
RST / VCF / VCW / VCB / RUN / FRP / BLK	PFU 实际状态编码，依次表示 reset、vector fetch、vector wait、vector branch、正常运行、force pending 和 blocked。
cache line	ICU 以固定大小管理的一段连续指令数据；PFU 用 32-byte 边界决定何时重新置 first，但不实现 cache 替换。
PF stage	RTL 使用的第一级名称。其英文展开在可读源码中未定义；本文按功能称为“取指地址/请求级”，负责地址选择、BTAC 查询、存储目标选择、MPU 查询和地址握手。
FE stage	RTL 使用的第二级名称。其英文展开在可读源码中未定义；本文按功能称为“取指响应/提交级”，负责等待数据、收集属性与错误，并提交 FIFO。
target path	分支被预测或实际判定为跳转后，从分支目标地址开始的指令路径。
fall-through path	分支不跳转时，紧随分支指令之后的顺序指令路径。
wrong path / correct path	错误预测后已经取回但必须丢弃的路径 / 与实际控制流一致、可以继续执行的路径。
taken / not-taken	分支实际或预测“跳转到目标” / “继续顺序执行”。
force	DPU 以 dpu_pfu_frc_v_i 和地址覆盖 PFU 当前程序流，使其从指定地址重新取指。
stall	某一级不能完成交接时保持有效事务和关联状态，不得重复提交或越过等待事务。
flush	清空旧程序路径上的 FIFO、预测元数据和待提交状态，使旧指令不再对 DPU 可见。
cancel	通知 ICU/TCU 忽略已发出但不再需要的地址或数据响应。
replay	已取指令因 TCU retry 或 ICU ECC 错误而需要重新取回和处理。
halfword	16-bit 半字，是 Thumb 指令流和 PFU FIFO 的基本存储单位。

1.2 宏观架构与主指令流



图中 cm7pfu 边界内明确画出了两个实际 RTL 子模块：cm7pfu_btac 位于地址预测旁路，cm7pfu_fifo 位于指令返回数据主路径。黑色实线表示 PF stage 向存储系统发出的取指地址，蓝色实线表示返回的指令数据或 BTAC 预测结果，灰色虚线表示 DPU 对 PFU 的消费和程序流反馈。

从地址路径看，cm7pfu 内部 PF stage 先选择当前取指地址，并用该地址并行查询 cm7pfu_btac。BTAC 命中且预测 taken 时，返回的目标偏移参与选择下一取指地址。地址落在 Instruction Tightly Coupled Memory (ITCM，指令紧耦合存储器) 或 Data Tightly Coupled Memory (DTCM，数据紧耦合存储器) 窗口时，请求送到 cm7tcu；其他地址送到 cm7icu。DPU 在分支结果确定后向 cm7pfu_btac 反馈 allocate、update 或 invalidate，使预测状态跟随实际执行结果更新。

从指令数据路径看，cm7icu 或 cm7tcu 最多返回 64-bit 指令块。FE stage 收集数据、有效范围、错误和调试保护属性，只在响应完整且未被取消时向 cm7pfu_fifo 提交。FIFO 把返回块压缩成连续的 16-bit 半字，识别 16-bit/32-bit Thumb 指令边界，组成 slot0 和 slot1，再送给 cm7pdu；DPU 用 pop 反馈本拍实际消费了零条、一条还是两条指令。这里的“双槽输出”只表示接口每周期最多提供两条指令，并不等价于两条指令一定能够同时发射；最终由 DPU 根据配对限制、数据相关和执行资源决定。

cm7pfu_btac 的命中、方向、entry index 和 offset 还通过 cm7pfu 内部元数据队列与对应 FIFO 指令保持同步。宏观图没有把这些 side queue 画成独立模块，因为它们是 cm7pfu 内部寄存器逻辑；其详细结构见 5.4。

1.3 核心职责与非目标

PFU 必须完成以下职责：

1. 维持 reset、向量取表、正常运行、force pending 和同步错误阻塞状态。
2. 按优先级选择向量地址、DPU force 地址、BTAC 目标地址或顺序地址。
3. 对新 32-byte 区域和非顺序访问发起 Memory Protection Unit (MPU，内存保护单元) 查询。
4. 在 ICU 与 TCU 之间选择唯一取指目标，处理 request/ack/data-valid、分段返回和 cancel。
5. 缓冲最多 16 个半字的物理数据，向 DPU 提供最多两条完整 Thumb 指令。
6. 预测分支并维持与指令严格对齐的 BTAC 元数据。

7. 把 breakpoint、MPU abort、bus error、vector error、retry/replay 转换为 DPU 可消费的错误语义。

PFU 不负责执行指令、计算最终分支条件、实现 ICU cache linefill/替换策略、实现 MPU region matching，也不决定 DPU 中哪些指令能够并行发射。

2. Module Composition

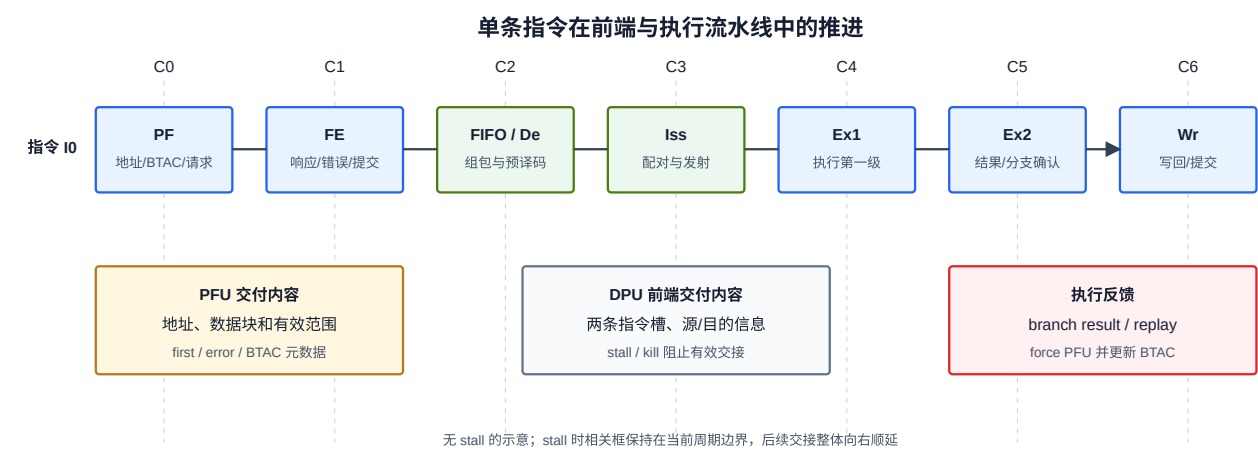
实际模块或内部功能块	职责	主要协作对象
cm7pfu	顶层状态机、PF/FE 两级管线、地址生成、TCM decode、请求/响应仲裁、异常与向量处理、flush/cancel。	cm7dpu、cm7icu、cm7tcu、MPU、Flash Patch and Breakpoint (FPB, Flash 补丁与断点) 接口、Nested Vectored Interrupt Controller (NVIC, 嵌套向量中断控制器)。
cm7pfu_fifo 实例 u_fifo	4-halfword 输入缓冲 + 12-halfword 主 FIFO；按半字压缩；组装两条 Thumb 指令；对齐 hit/error/protection。	FE stage 和 DPU instruction/pop 接口。
cm7pfu_btac 实例 u_btac	4-bank Branch Target Address Cache (BTAC, 分支目标地址缓存) 查询、方向预测、轮转分配、更新、失效和 multiple-hit 自修复。	PF 地址生成和 DPU BTAC maintenance。
cm7pfu 内部 BTAC side queues	保存每个命中半字的 index/taken, 以及每个 taken fetch 的 offset/off_x, 使预测信息与 FIFO 指令同步。	u_btac、u_fifo、DPU。
cm7pfu 内部 exception logic	按半字计算 breakpoint/MPU/bus/vector/replay 范围和统一错误码。	FE stage、u_fifo、DPU。

cm7pfu_fifo 和 cm7pfu_btac 是真实 RTL module；地址选择、异常编码、side queue 和主状态机是 cm7pfu 内部逻辑，不应在重实现中误写成外部独立模块接口。

3. Pipeline And Branch Prediction Primer

3.1 通用流水线概念

流水线把一条指令的处理拆到多个周期，使不同指令可以在不同阶段重叠。PFU 只直接实现 PF 和 FE 两级，但其输出进入 DPU 后还会经历 decode、issue、execute 和 write/retire。valid 表示某个框中确实有一条有效指令；stall 使该阶段及必要的上游保持；flush/kill 使错误路径的 valid 失效。



该图每一列代表一个 cycle，每个框都吸附到周期列。PF stage 输入当前程序流地址、force/vector 候选和 BTAC 结果，输出存储请求地址、目标选择以及 MPU 查询上下文。FE stage 输入 memory response、MPU 属性和错误，输出 64-bit fetch block 及其有效半字范围。PFU FIFO 把块转换为最多两个 instruction slot，DPU decode 识别操作类型和寄存器需求，issue 决定槽位能否进入执行，后续 execute/write 确认分支结果并产生必要的 force 或 BTAC 更新。

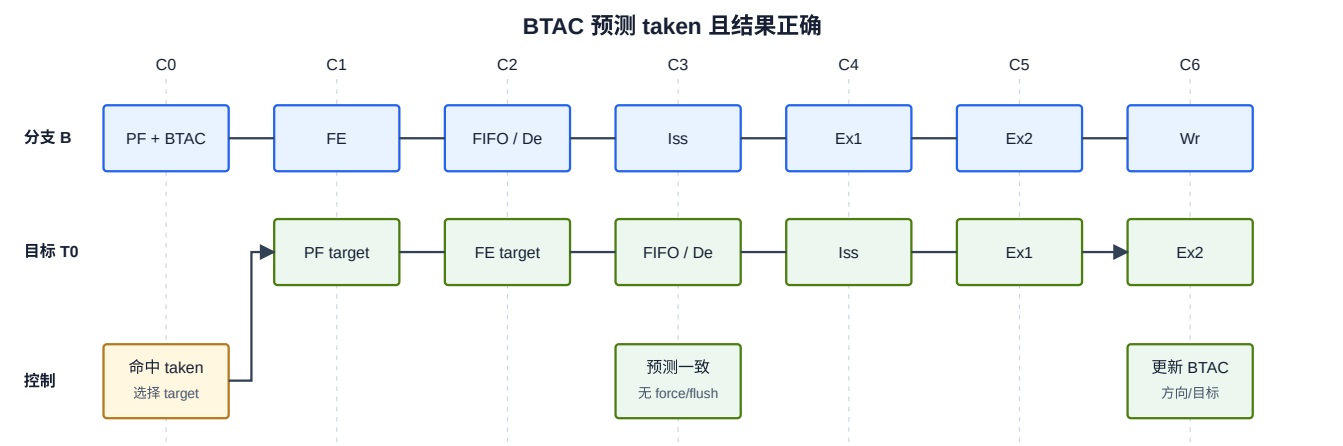
PF stall 时，地址事务和 `addr_fe/v_fe` 交接必须保持；FE stall 时不能覆盖尚未完成的数据事务。DPU 不 pop 时，FIFO 输出保持同一顺序指令。force、interrupt 或同步异常发生后，旧路径框即使已经经过 PF/FE，也必须被 flush/kill，不得继续产生架构可见效果。

3.2 PFU 内部 PF/FE 两级交接

Stage	输入	本级处理	输出给下一级
PF	状态机、 <code>addr_pf_q</code> 、vector PC、DPU force、BTAC hit/offset、TCM 配置、FIFO full。	选择 <code>addr_pf</code> ；判断 ITCM/DTCM/ICU；生成 request、first、privilege、vector-fetch 属性；并行发 MPU lookup；等待 address ack。	ack 后把地址、目标选择和有效事务写入 <code>addr_fe/cs_fe/v_fe</code> 。同时计算顺序地址或预测 target，更新 <code>addr_pf_q</code> 。
FE	<code>addr_fe/cs_fe/v_fe</code> 、ICU/TCU 数据与错误、MPU abort/attributes、FPB hit。	等待完整 data phase；选择 64-bit 数据源；计算有效半字、错误范围和代码；处理 fake data、cancel、retry/replay。	正常时 <code>fifo_push</code> 到 <code>cm7pfu_fifo</code> ；向 side queue 写 BTAC 元数据；向 DPU 产生 vector/MSP 或 instruction/error。

PF 可在前一条请求处于 FE data phase 时发出下一地址，从而重叠地址和数据阶段。但当待返回数据来自一个 target，而新地址要转到另一个 target 时，PFU 必须取消冲突事务，不能把 ICU 数据与 TCU 地址或相反组合成同一个 fetch。

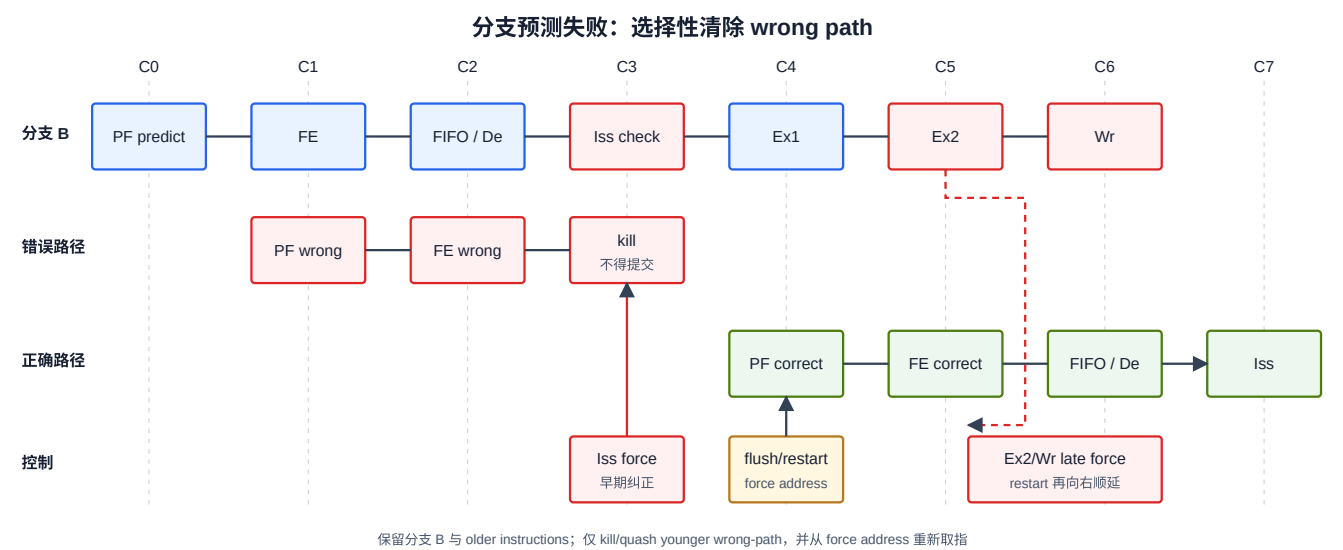
3.3 预测命中且结果正确



分支指令在 PF 查询时命中 BTAC 且方向位预测 taken，下一 PF 周期直接请求 target path。分支本身随后经过 FE、FIFO、decode 和 issue；与此同时目标指令已经在前端流动，所以分支实际结果与预测一致时不需要清空流水线。DPU 仍会在结果确定后更新 taken counter，并在目标偏移变化时更新 target。

图中目标指令比“等待执行后再取目标”的方案更早进入 PF/FE。每一级向下一级交付的不只是 instruction bits，还包括 valid、Thumb size、BTAC hit/index/offset、error 和 protection。任一级 stall 都会把后续交接整体右移；正确预测本身不会产生 flush。

3.4 预测失败与恢复



预测失败包括方向错误和目标错误。预测 taken 但实际 not-taken 时，target path 是 wrong path，正确地址是 fall-through；预测 not-taken 但实际 taken 时，已取的 fall-through 是 wrong path；预测 taken 且实际也 taken，但目标地址不同，仍属于 target mispredict。

DPU 可在 issue、Ex2 或 Wr 阶段产生 force。更晚阶段代表更老、更接近提交的指令，因此 force 地址优先级固定为 `Wr > Ex2 > Iss`；Wr 内 slot0 高于 slot1，且 replay 高于同槽 branch force。PFU 收到 force 的同周期产生 flush/cancel：FIFO、BTAC side queues 和旧 FE response 被屏蔽，然后从 force address 重新开始。若 PF 正在等待 ack/data，状态进入 `FRP` 保存 pending force，直到旧事务可安全取消或 stall 解除。

这里必须区分“逻辑 flush 错误路径”和“复位整条流水线”。分支预测失败需要前者，但不需要后者。处理器是按程序顺序提交的：当分支 B 在 Iss、Ex2 或 Wr 被确认预测错误时，B 之前的指令更老，已经属于确定的正确历史；B 本身也必须保留，因为它产生实际 taken/not-taken 和正确 target。只有 B 之后、沿预测方向进入流水线的 younger instructions（更年轻指令）属于 wrong path。

位置	预测失败后的动作	为什么可以这样处理
outstanding ICU/TCU request、PF/FE transaction	cancel 或使 valid 失效。	它们是分支后继续预取的内容，尚未形成架构结果。
<code>cm7pfu_fifo</code> 的 12-halfword 主 FIFO	<code>fifo_n=0</code> 、 <code>fifo_wp=0</code> 、 <code>fifo_rp=entry0</code> ，所有旧输出变为 invalid。	分支已被 DPU pop；主 FIFO 中尚未消费的指令全部比分支年轻。
<code>cm7pfu_fifo</code> 的 4-halfword <code>imem</code> 入口缓冲	<code>imem_n=0</code> 、 <code>imem_rp=0</code> ，同拍 push/pop 也不能留下有效内容。	<code>imem</code> 保存的是尚未搬入主 FIFO 的更年轻取指返回，同样属于预测路径。
BTAC side queues	读写指针恢复为空队列状态，丢弃与 wrong-path 指令对应的 index/taken/offset 元数据。	指令已清空后，其预测元数据也必须同时清空，否则会与恢复后的新指令错配。

位置	预测失败后的动作	为什么可以这样处理
cm7pfu_btac 预测表	不整体清空；DPU 根据失败原因 update 或 invalidate 产生错误预测的 entry。	其他 entry 保存的是无关分支历史，全部清除会损失有效预测信息。
ICU Cache、ITCM、DTCM和外部存储器	不清除存储内容，只取消错误路径的未完成地址或数据事务。	分支预测失败改变的是程序流，不表示存储器中的指令数据失效。
DPU 同拍双槽	slot0 分支要求 force 时，br_quash_iss[1] / 对应后级 quash 使 slot1 失效。	slot1 在程序顺序上位于 slot0 分支之后，必然是 younger wrong-path instruction。
DPU 较年轻 stage	从 mispredict 所在 stage 向前端级联 kill；被 kill/quash 的 valid、PC 更新、寄存器写回和 LSU side effect 必须被屏蔽。	stage 位置编码了指令年龄，可定向删除 wrong path，不必清零所有 pipeline register。
mispredicting branch B 与更老指令	保留，允许继续到 Wr/retire。	它们属于正确历史；删除它们既没有必要，也会重复已经完成的工作。

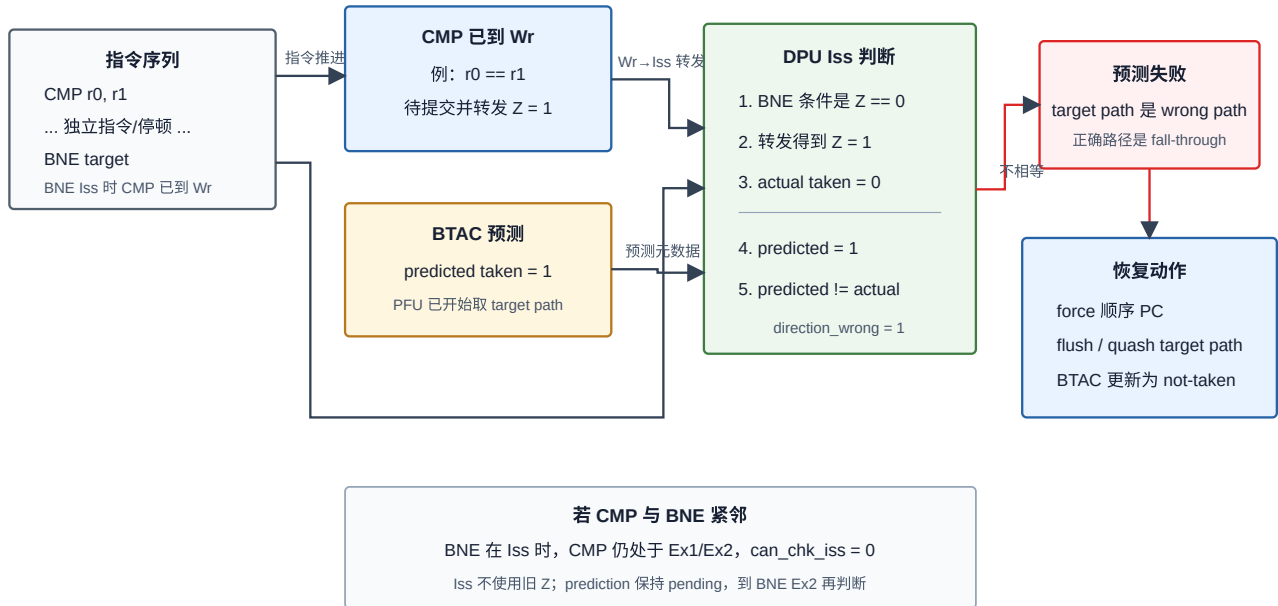
cm7pfu_fifo 的“清空”是功能性清空：valid、count 和 pointer 恢复为空状态，imem_d 与主 FIFO 存储阵列中的旧数据位不要求全部写成 0。只要这些旧位没有 valid，就不能再次被组装为指令。PFU 同时令 address/data cancel 有效，所以错误路径的 ICU/TCU 响应即使稍后返回，也不能重新 push 到已经清空的 FIFO。

所以图中的 kill 不是“什么都不清”，而是精确地把 wrong-path valid 置为无效；flush 也不是清除Cache或TCM，而是清除PFU内部错误路径状态。数据位可以继续留在流水寄存器或FIFO存储阵列中，只要valid/kill/quash阻止它产生任何架构可见副作用。这样既保证正确性，也缩短从force address恢复的时间。

3.5 BNE 在 Iss 阶段判断的例子

Branch if Not Equal (BNE，不相等则跳转) 在 z=0 时 taken，在 z=1 时 not-taken。下面固定一个能够在 Iss 提前判断的方向预测失败例子：CMP r0,r1 与 BNE target 之间存在足够的独立指令或流水停顿，使 BNE 位于 Iss 时 CMP 已经到 Wr。CMP 的比较结果相等，Wr 正准备提交 z=1；BTAC 却预测 BNE taken。

BNE 使用 Wr forwarding 在 Iss 发现预测错误



读图顺序是左侧指令和两个输入、中间 Iss 判断、右侧恢复动作。CMP 不保存减法结果，只产生 NZCV；普通 ALU/compare 的候选标志在 Ex2 计算，架构 `nzcv_ret` 在 Wr 提交。这里 Iss 并不是提前读取“未来值”，而是通过 Wr → Iss forwarding 选择 Wr 正准备提交的 `new_nzcv_wr`；若 Wr 本拍不更新，则读取已经提交的 `nzcv_ret`。当 `r0==r1` 时转发值为 `Z=1`，所以 BNE 的实际方向是 not-taken。BTAC 元数据却表示 predicted taken，两者不相等，形成 direction mispredict。

Iss 判断步骤	已知信息	结果
1. 读取预测	BTAC hit, <code>btac_taken_iss=1</code> ，PFU 已沿 target path 取指。	predicted direction = taken。
2. 取得条件标志	<code>nzcv_0_ex2 = nzcv_set_wr ? new_nzcv_wr : nzcv_ret</code> ，且没有 Ex1/Ex2 中更老的待更新 flag。	Wr forwarding 提供有效 <code>Z=1</code> ， <code>can_chk_iss=1</code> 。
3. 计算实际方向	指令是 BNE，成立条件为 <code>Z==0</code> ；当前 <code>Z=1</code> 。	actual direction = not-taken。
4. 比较	<code>predicted_taken != actual_taken</code> 。	<code>dir_wrong_iss=1</code> 。
5. 生成恢复地址	实际 not-taken。	force address 选择 BNE 后的 sequential/fall-through PC。
6. 清理与维护	target path 已经进入 PFU/DPU 前端。	flush PFU、quash younger wrong-path，并用实际 not-taken 更新 BTAC。

这个例子能够在 Iss 判断的前提是 NZCV 已经有效：Ex1/Ex2 中没有更老的未完成 flag-setting instruction，且 Wr 中的普通 flag 结果能够被 forwarding。`nzcv_ex2_v_iss` 专门检查 Iss 前方是否仍有这种未完成更新。

若 CMP 与 BNE 紧邻，无 stall 的典型推进如下。此时 BNE 在 Iss **不能**判断预测正确性：

Cycle	CMP	BNE	分支处理
C0	Iss	-	CMP 被识别为 flag-setting instruction。
C1	Ex1	Iss	Ex1 中有更老的 flag writer，nzc_v_ex2_v_iss=0、can_chk_iss=0；BNE prediction 标记为 pending。
C2	Ex2，产生候选 NZCV	Ex1	不在 Iss 使用尚未产生的 Z。
C3	Wr，提交/转发 NZCV	Ex2	BNE 在 Ex2 使用正确 NZCV 完成方向和 target 检查。

因此实现合同不是“所有 BNE 都在 Iss 判断”，而是“只在 can_chk_iss=1 时允许 Iss 判断；否则必须保留 prediction context 并延迟到 Ex2/Wr”。

4. External Interface Contract

4.1 Clock、Reset 与配置

端口	方向/位宽	有效与保持合同
clk	in, 1	所有功能状态在上升沿更新。
reset_n	in, 1	异步低有效复位；主状态回 RST，FIFO/BTAC valid 清空，输出 valid 失效。
cpuwait_i	in, 1	在 RST 为 1 时保持 reset 状态。
miu_prod_mbist_en_i	in, 1	MBIST 期间保持 RST。
itcm_en_i/dtcm_en_i	in, 1/1	控制 TCM 地址 decode；变化时下一 fetch 必须标记 first。
itcm_size_i/dtcm_size_i	in, 4/4	生成 TCM region mask；size 为 0 时对应 TCM 不命中。
btac_en_i[1:0]	in, 2	bit0 使能 lookup/hit；bit1 使能 allocation。update/invalidate 按各自控制执行。
bigend_i	in, 1	只对 vector/MSP 32-bit word 做 byte swizzle，不改变普通 Thumb FIFO 顺序。

4.2 DPU 指令与程序流接口

端口组	方向	合同
pfu_dpu_inst_iss_o[63:0]	out	{slot1[31:0], slot0[31:0]}；16-bit 指令仍占一个 32-bit slot，size 单独指示。
pfu_dpu_inst_v_iss_o[1:0]	out	每槽 valid；slot1 不得在 slot0 无效时被独立消费。

端口组	方向	合同
<code>pfu_dpu_inst_t32_iss_o[1:0]</code>	out	1 表示 32-bit Thumb, 0 表示 16-bit Thumb。
<code>pfu_dpu_err_iss_o[1:0]</code> / <code>pfu_dpu_err_code_o[3:0]</code>	out	error 与指令槽对齐；同一未消费错误指令期间 code 保持稳定。
<code>pfu_dpu_prot_iss_o[1:0]</code>	out	debug instruction protection, 32-bit 指令要求两个半字 protection 均有效。
<code>dpu_pfu_pop_iss_i[1:0]</code>	in	00 不消费, 01 消费 slot0, 11 消费两槽；10 非法。仅 valid 槽实际生效。
<code>dpu_pfu_frc_v_i/addr_i</code>	in	force valid 同周期覆盖地址 mux；stall 时必须保存 pending force。force 只允许在 RUN/FRP/BLK/VCB 合法上下文出现。
<code>dpu_pfu_stop_i</code>	in	请求停止普通 fetch；stall 中到达时由 <code>blp_v</code> 记忆。force 高于 stop。
<code>dpu_pfu_int_v_i</code>	in	请求新的异常向量取表；reset vector 尚未完成时不得输入普通 interrupt。
<code>dpu_pfu_priv_ctl_i[1:0]</code>	in	bit0 为更新使能, bit1 为新 privilege；interrupt 强制 privilege=1。
<code>dpu_pfu_nmihf_i</code>	in	指示当前是否处于 NMI/HardFault 上下文；PFU 每拍保存并送 MPU。
<code>pfu_dpu_idle_o</code>	out	<code>RUN & (fifo_full !bt_fe_ok)</code> 或 BLK。它表示 PFU 无法/无需继续预取, 不表示整个 core 无活动。
<code>pfu_in_reset_o</code>	out	<code>state_pf==RST</code> 时有效。

4.3 Vector、BTAC 与维护接口

端口组	方向/位宽	合同
<code>pfu_dpu_vect_v_o</code>	out, 1	vector handoff valid。
<code>pfu_dpu_vect_a_o</code>	out, 32	vector PC；vector error 时按 RTL 置为全 1。
<code>pfu_dpu_vect_r_o/e_o</code>	out, 1/1	<code>r</code> 表示 vector replay, <code>e</code> 表示 vector error。
<code>pfu_dpu_msp_v_o</code>	out, 1	reset vector handoff 同拍给出 MSP valid；普通异常保持 0。
<code>pfu_dpu_msp_d_o</code>	out, [31:2]	初始 MSP；vector error 时按 RTL 置为全 1。
<code>pfu_dpu_btac_hit_o</code>	out, 4	两个 instruction slot 各携带两个 halfword hit bit。
<code>pfu_dpu_btac_taken_o</code>	out, 1	与当前最早被消费的 BTAC hit 指令对齐。
<code>pfu_dpu_btac_index_o</code>	out, 4	与 taken/index side queue 头部对齐。
<code>pfu_dpu_btac_offset_o</code>	out, 12	与 taken branch 的 offset queue 头部对齐。
<code>pfu_dpu_btac_off_x_o</code>	out, 1	adjusted negative offset 发生符号越界, 需要 DPU refetch。

端口组	方向/位宽	合同
<code>dpu_btac_alloc_i</code>	in, 1	为未命中的有效分支在其 bank 当前 <code>wr_pt</code> 分配 entry。
<code>dpu_btac_update_i</code>	in, 2	bit0 更新方向；bit1 与 bit0 配合做 full update，同时改 tag 和 offset。
<code>dpu_btac_inv_i</code>	in, 2	bit0 失效当前 lookup 命中并屏蔽 hit；bit1 延续到下一 lookup，且必须与 bit0 同时输入。
<code>dpu_btac_addr_i</code>	in, [31:1]	allocation/update 对应 branch instruction 地址，同时选择 bank。
<code>dpu_btac_index_i</code>	in, 4	update 的 bank 内 entry index。
<code>dpu_btac_taken_i</code>	in, 1	branch 实际方向。
<code>dpu_btac_offset_i</code>	in, 12	full update/allocation 使用的新 target offset。

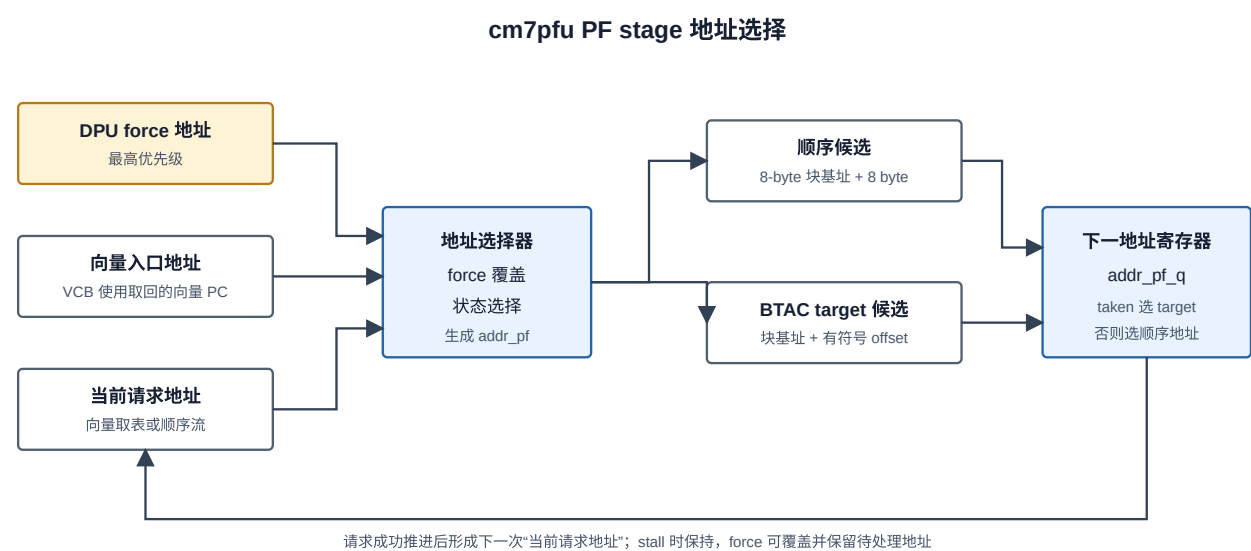
4.4 ICU、TCU、MPU、FPB 与 NVIC

端口	方向/位宽	有效与保持合同
<code>nvic_int_nxt_isr_i</code>	in, 8	exception vector number；仅 vector fetch 地址形成时采样。
<code>ppb_vto_tbloff_i</code>	in, [31:7]	Vector Table Offset Register (VTOR，向量表偏移寄存器) 基址；与 vector number 组合。
<code>ppb_dbg_vcatch_i</code>	in, 1	普通 exception 的 debug vector-catch 请求。
<code>ppb_dbg_rst_vc_i</code>	in, 1	reset 的 debug vector-catch 请求。
<code>mpu_pfu_abort_p1_i</code>	in, 1	与前一 P0 lookup 对应；abort 生成 fake data 和 code <code>0010</code> 。
<code>mpu_pfu_attribs_p1_i</code>	in, 4	与前一 lookup 对应；送到 ICU 并与 fetch transaction 关联。
<code>pfu_mpu_valid_p0_o</code>	out, 1	<code>v_pf && first_pf</code> 时有效。
<code>pfu_mpu_addr_p0_o</code>	out, [31:5]	当前 PF fetch 的 32-byte region 地址。
<code>pfu_mpu_priv_p0_o</code>	out, 1	当前 privilege；interrupt vector 强制 privileged。
<code>pfu_mpu_vf_p0_o</code>	out, 1	<code>VCF</code> vector-table fetch 标记。
<code>pfu_mpu_nmihf_o</code>	out, 1	当前 NMI/HardFault 上下文。
<code>pfu_tcu_req_o</code>	out, 1	有效 PF transaction 且 speculative decode 选择 TCM。ack/cancel 前保持事务语义。
<code>pfu_tcu_target_o</code>	out, 1	0 表示 ITCM，1 表示 DTCM。
<code>pfu_tcu_addr_o</code>	out, [23:3]	8-byte fetch block 地址。
<code>pfu_tcu_priv_o</code>	out, 1	当前 fetch privilege。
<code>pfu_tcu_vf_o</code>	out, 1	当前请求是 VCF vector-table fetch。
<code>pfu_tcu_cancel_resp_o</code>	out, 1	丢弃当前/后续 data response；被取消 dvalid 不得写 FIFO。

端口	方向/位宽	有效与保持合同
<code>pfu_tcu_cancel_last_addr_o</code>	out, 1	取消最后一个 address phase，覆盖 address mispredict 和 target overlap。
<code>tcu_pfu_ack_i</code>	in, 1	与 request/address 同周期采样；0 使 PF address stall。
<code>tcu_pfu_dvalid_i</code>	in, 2	bit0/bit1 分别写入低/高 32-bit word。
<code>tcu_pfu_data_complete_i</code>	in, 1	整个 fetch response 已完成；它而不是单个 dvalid 解除 FE data wait。
<code>tcu_pfu_err_i</code>	in, 1	与完成响应对齐的 bus/error indication。
<code>tcu_pfu_data_i</code>	in, 64	dvalid 对应 word 有效；完整块在 complete 后可提交。
<code>tcu_pfu_retry_i</code>	in, 1	对上一已 push fetch 触发 replay。
<code>pfu_icu_req_o</code>	out, 1	有效 PF transaction 且未选择 TCM；ack/cancel 前保持事务语义。
<code>pfu_icu_addr_o</code>	out, [31:3]	8-byte fetch block 地址。
<code>pfu_icu_attrs_o</code>	out, 4	MPU P1 返回的 memory attributes。
<code>pfu_icu_first_o</code>	out, 1	非顺序或新的 32-byte line/stream 起点。
<code>pfu_icu_priv_o</code>	out, 1	当前 fetch privilege。
<code>pfu_icu_vf_o</code>	out, 1	当前请求是 VCF vector-table fetch。
<code>pfu_icu_cancel_resp_o</code>	out, 1	丢弃 outstanding ICU response，并屏蔽 FIFO write。
<code>icu_pfu_ack_i</code>	in, 1	与 request/address 同周期采样；0 使 PF address stall。
<code>icu_pfu_dvalid_i</code>	in, 1	<code>icu_pfu_data_i</code> 的 64-bit response 有效。
<code>icu_pfu_data_i</code>	in, 64	ICU 返回的完整 fetch block。
<code>icu_pfu_bus_err_i</code>	in, 1	与 dvalid/response 对齐，生成 code 0011。
<code>icu_pfu_ecc_err_i</code>	in, 1	对上一已 push fetch 触发 replay。
<code>fpb_pfu_bkpt_match_i</code>	in, 4	每个 halfword 一个 breakpoint match。
<code>pfu_fpb_inst_addr_o</code>	out, [31:3]	FE fetch block 地址。
<code>pfu_fpb_inst_addr_valid_o</code>	out, 1	<code>v_fe</code> ，表示 FE 地址可供 FPB 比较。
<code>dbg_iprot_i</code>	in, 1	屏蔽 FPB breakpoint，并作为 per-halfword protection 写入 FIFO。
<code>dbg_iaddr_o</code>	out, [31:3]	当前 FE instruction block 地址，供 debug protection 使用。

5. Data Model And Address Generation

5.1 地址选择与优先级



`addr_pf` 是本周期对 memory system 可见的请求地址。DPU force 对组合地址 mux 具有最高覆盖权；否则 `actl_pf` 在三个来源中选择 `addr_pf_q`、`addr_fe` 或 vector PC。请求推进后，`addr_pf_q` 保存下一候选地址：BTAC taken 使用目标偏移，其他情况使用顺序地址。

顺序地址表达式以当前 8-byte fetch block 的基址为准加 8 byte，即 `align_down(addr_pf, 8) + 8`。RTL 地址总线省略 bit0，因此 `[31:1]` 数值加 4 对应字节地址加 8；重实现不能误写成加 4 byte。BTAC target 为 `align_down(addr_bt, 8) + sign_extend(offset[12:1])`。DPU force 与 stall 同时出现时，force 地址直接成为待保存的下一地址，避免被旧请求覆盖。

地址来源优先级如下：

优先级	条件	本周期 <code>addr_pf</code> / 下一地址行为
1	<code>dpu_pfu_frc_v_i</code>	立即使用 force address；同时 flush 旧路径。
2	VCB 需要从 vector PC 分支	使用 vector table 返回的 PC。
3	VCF/VCB 的向量取表序列	使用保存的 vector table address。
4	有效 BTAC taken	下一候选使用 target。
5	普通 RUN	下一候选使用顺序地址。

`first_pf` 必须在非顺序请求、跨 32-byte line、TCM decode 预测纠正、BTAC side queue full 后恢复、TCM enable 变化、taken target 不在同一 line 时置位。任何 `first_pf=0` 的有效请求必须仍位于上次 first lookup 的同一 `[31:5]` 区域。

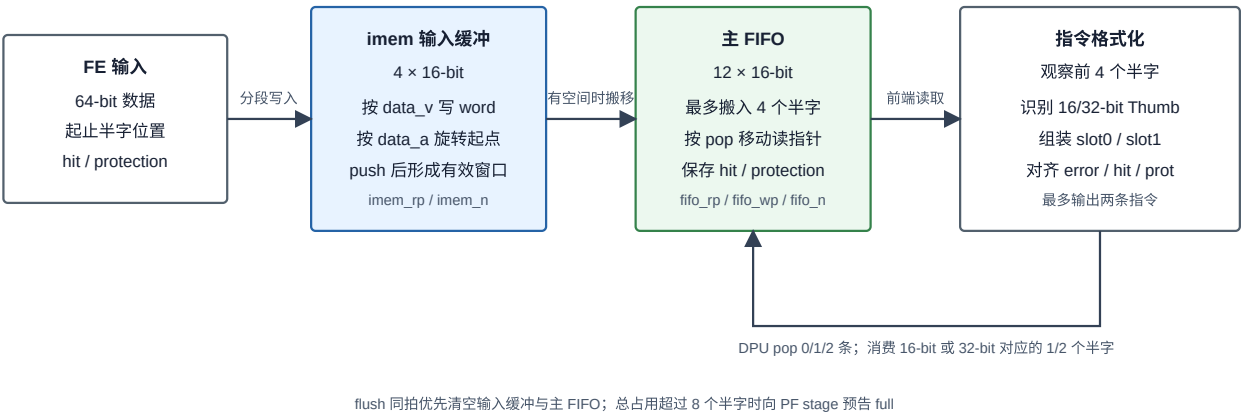
5.2 Prefetch Queue (PFQ) 结构

Prefetch Queue (PFQ，预取队列) 是取指存储器返回接口与 Data Processing Unit (DPU，数据处理单元) 指令接口之间的弹性缓冲。它不只是保存数据，还负责完成以下工作：把不对齐的 64-bit 返回块压缩成连续半字流；识别 Thumb 16-bit/32-bit 指令边界；同时形成两个有序指令槽；为每个半字保存与自身对应的伴随属性。Branch Target Address Cache (BTAC，分支目标地址缓存) 命中来自分支预测查询，错误由 PFU 汇总取指异常后生成，调试保护来自外部 IADBGPROT 输入；半字在缓冲区中搬移或组成指令时，这些属性必须始终与对应数据保持对齐。

5.2.1 物理组成与逻辑视图

PFQ 在 RTL 注释中描述为 16×16-bit，物理上由 cm7pfu_fifo 内的 4-halfword imem 输入缓冲和 12-halfword 主 FIFO 组成。Halfword（半字）在本文中固定指 16 bit，也是 Thumb 指令打包和 FIFO 计数的最小单位。

cm7pfu_fifo：16 个半字的压缩与双指令输出



图中的 4+12 是物理存储划分，不是两个软件可见队列。指令格式化逻辑把主 FIFO 队首和尚未搬入的 imem 内容合并成一条连续半字流，DPU 看不到数据来自哪个物理区域。主 FIFO 用循环读写指针避免每次 pop 时移动全部存储项。

imem 这个名字容易被误解。它不是 Instruction Memory（指令存储器），也不是 instruction cache；在本模块中，它是位于主 FIFO 入口处的 memory-data staging buffer（存储器返回数据暂存缓冲），一次最多保存一个 64-bit fetch block。其状态和内容为：

状态/存储	含义
imem_d[0..3]	四个 16-bit 数据寄存器，合计保存一笔 64-bit 返回。
imem_rp	当前有效窗口的第一个半字位置，由 data_a_i 建立。
imem_n	当前有效窗口包含的半字数量，范围为 0..4。
ihit_d[3:0]	四个物理半字各自的 BTAC hit 属性。
iprt_d[3:0]	四个物理半字各自的调试保护属性。

imem 有四项不可省略的功能：

- 1. 吸收分段返回。Tightly Coupled Memory Unit (TCU，紧耦合存储单元) 可以先返回低 32 bit、后返回高 32 bit。imem_d 分别保存两个 word，直到 push_i 确认整块已经收齐；主 FIFO 不会看到半个尚未完成的 fetch block。

2. **对齐并裁剪有效范围。** 存储器返回的是对齐的 64-bit block，但真正需要的指令可以从其中任意半字开始，也可能因同一 block 内预测 taken 而提前结束。imem_rp 和 imem_n 只暴露 data_a_i..data_e_i 之间的连续半字，块前后的数据不进入逻辑指令流。
3. **吸收主 FIFO 背压。** 当 12-entry 主 FIFO 暂时不能接收新的四个半字时，已经返回的 block 可以停留在 imem，不必覆盖主 FIFO 或丢弃存储器响应；主 FIFO 释放空间后，imem 的剩余有效半字整体搬入。
4. **提供低延迟旁路读取。** 指令格式化器可以直接读取 imem，不要求数据先搬入主 FIFO。若一条 32-bit 指令的第一半在主 FIFO、第二半在 imem，两个区域仍能在同一逻辑读窗口中拼成完整指令。

“指令可以从 block 中任意半字开始”的原因是存储器返回粒度和指令地址对齐粒度不同。存储器接口返回按 8 byte 边界对齐的 64-bit block，而 Cortex-M7 的 Thumb 指令首地址只要求按 2 byte 半字边界对齐。因此，同一个 64-bit block 中有四个都可能成为合法指令起点的半字位置：

addr_fe[2:1]	相对 64-bit block 基地址的偏移	data_a_i/imem_rp 选择
00	0 byte	HW0
01	2 byte	HW1
10	4 byte	HW2
11	6 byte	HW3

例如，分支、函数返回、异常返回或 DPU force 给出的目标地址是 0x1006。存储器仍返回以 0x1000 为基地址的 {HW0, HW1, HW2, HW3} 整块数据，但 addr_fe[2:1]=2'b11，所以 data_a_i=3、imem_rp=3，只有从 HW3 开始的数据属于新取指路径；HW0..HW2 位于目标地址之前，必须被有效窗口裁掉。这里的“任意半字”只表示目标指令可能落在四个半字槽位中的任意一个，目标地址仍必须指向真实指令边界，不能指向一条 32-bit 指令的第二个半字。

一笔返回在 imem 中的生命周期如下：data_v_i 分拍写入原始数据；push_i 建立 imem_rp/imem_n 和伴随属性；随后 DPU 可以直接读取其中的有效半字；主 FIFO 有空间时，尚未被直接消费的半字一次性搬入主 FIFO 并清空 imem_n。因此，imem 同时解决“返回可能分拍到达”“返回块可能不对齐”和“主 FIFO 可能暂时没有入口空间”三个问题。

5.2.2 数据写入与 block 提交

前端返回分为“写数据”和“提交有效范围”两个动作，二者不能混为一个握手：

输入	精确功能
data_v_i[0]	本拍 data_d_i[31:0] 有效，写入 imem_d[0]、imem_d[1]。
data_v_i[1]	本拍 data_d_i[63:32] 有效，写入 imem_d[2]、imem_d[3]。
push_i	整个 64-bit fetch block 已收齐且允许进入 PFQ；在该上升沿建立新的有效半字窗口。它不是第三份数据。
data_a_i	block 中第一个有效半字的 index，取值 0..3，并成为新的 imem_rp。
data_e_i	block 中最后一个有效半字的 index，取值 0..3。正常窗口的有效数为 data_e_i-data_a_i+1。
data_h_i[3:0]	每个物理半字各一位的 BTAC hit；在 push_i 时与 block 一起提交。

输入	精确功能
<code>data_p_i[3:0]</code>	每个物理半字各一位的 debug protection；在 <code>push_i</code> 时与 block 一起提交。

TCU 的低、高 32-bit 可以在不同 cycle 到达：先到的 word 只写入 `imem_d`，不会增加队列有效计数；最后一个 word 到达时可以同拍拉起 `push_i`。非 flush 的 `push_i` 必须保证两个 word 已在本拍或此前到达。提交后，`imem_n` 取 0..4；若 `data_e_i < data_a_i`，硬件把有效数饱和为 0，而不是产生环绕窗口。正常取指事务应给出连续且非倒序的范围。

这套两阶段语义很重要：实现不能在第一次 `data_v_i` 到达时就向 DPU 暴露指令，否则 split response 的后一半尚未到达，错误范围、末尾位置和 BTAC 元数据也尚未最终确定。

5.2.3 连续半字读窗口

读取端总是组合观察“主 FIFO 的头部 + imem 尚未搬入的内容”中的前四个 halfword，因此可在跨物理边界时组装 32-bit Thumb。

Thumb 指令流以 16-bit 半字为基本排列单位，但一条指令可能占一个或两个半字。对于当前队首半字，PFU 检查最高五位 `[15:11]`：

<code>[15:11]</code> 前缀	PFU 对指令长度的解释	后续动作
<code>11101</code> 、 <code>11110</code> 、 <code>11111</code>	这是 32-bit Thumb 指令的第一个半字，当前半字本身不是完整指令。	必须等待紧随其后的第二个半字；两个半字共同形成一个 slot，pop 时消费两个半字。
其他前缀	这是完整的 16-bit Thumb 指令。	当前半字单独形成一个 slot，pop 时只消费一个半字。

RTL 中的 `instr[15:13]==3'b111 && instr[12:11]!=2'b00` 正是在压缩上述三个 5-bit 前缀：第一部分要求前缀以 `111` 开头，第二部分排除 `11100`，最终只留下 `11101/11110/11111`。这个判断不是对指令功能或操作码类别的完整译码，只用于确定当前指令占一个还是两个半字。

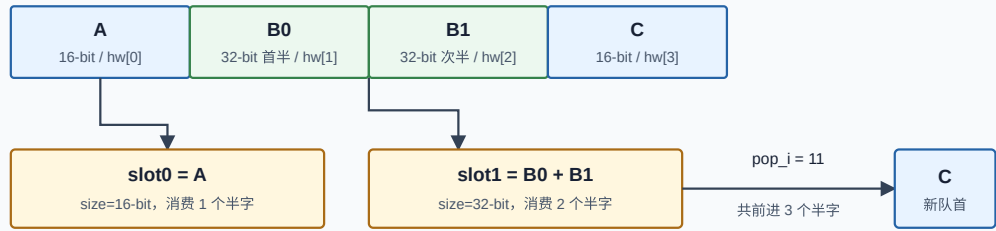
长度判断直接控制三个行为。第一，若判定为 32-bit 而第二半字尚未有效，slot 必须保持 invalid，不能把半条指令交给 DPU；错误尾部强制推进是唯一例外。第二，slot0 为 16-bit 时 slot1 从 `hw[1]` 开始，slot0 为 32-bit 时 slot1 必须跳过第二半，从 `hw[2]` 开始。第三，DPU pop 一条指令时，内部读指针根据长度前进一个或两个半字。因此，重实现不能只复制布尔表达式，还必须保持它对 valid、slot 边界和消费数量的全部影响。

逻辑半字 `hw[0]..hw[3]` 按以下规则形成：先取主 FIFO 从 `fifo_rp` 开始的最多四项；若主 FIFO 不足四项，再按 `imem_rp` 从输入缓冲补足。每个 `hw[k]` 同时携带 `{data, first-half, BTAC-hit, protection, error, valid}`，不能只拼接 data 而从另一套 index 读取属性。

cm7pfu_fifo 半字打包与跨物理边界读取

A. 两个指令槽与半字消费

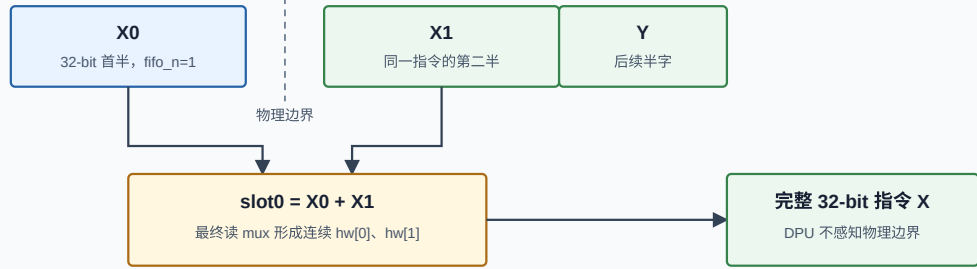
逻辑读窗口（队首在左）



蓝色表示 16-bit 指令；绿色相邻两格共同构成一条 32-bit 指令。

B. 32-bit 指令跨主 FIFO 与 imem 边界

12-entry 主 FIFO 队首



data、BTAC hit、protection 和 error 都沿相同半字选择路径拼接。

图 A 展示半字级打包。队首依次为 16-bit 指令 A、32-bit 指令 B 的两个半字 B0/B1、16-bit 指令 C。格式化器首先输出 slot0=A；因为 A 只占一个半字，slot1 从 hw[1] 开始并输出 B0+B1。DPU 给出 pop_i=11 后，A 和 B 共消费三个半字，C 成为下一拍的 hw[0]。因此 pop 的单位在接口上是“指令条数”，内部指针移动单位却是“半字数”。

图 B 展示物理边界对指令不可见。32-bit 指令 X 的第一半 X0 位于 12-entry 主 FIFO 队首，第二半 X1 尚在 4-entry imem。最终读 mux 把两者放到相邻的 hw[0]、hw[1]，slot0 仍输出完整 X。重实现若要求 32-bit 指令的两个半字必须位于同一存储区，会在这个场景错误停顿或破坏指令。

5.2.4 两个指令槽的形成规则

这里的 slot0 和 slot1 不是 FIFO 中两个固定的存储项，而是 cm7pfu_fifo 每拍从队首连续半字流中组合形成的两个指令视图。slot0 永远表示最老、下一条应被 DPU 处理的指令；slot1 表示紧跟在 slot0 后面的下一条指令。slot1 的起始位置不是固定的，必须先知道 slot0 占一个还是两个半字。

形成两个 slot 的步骤如下：

1. **确定 slot0 起点。** slot0 固定从队首 hw[0] 开始。
2. **确定 slot0 长度。** 若 hw[0] 的前缀表示 16-bit Thumb，slot0 只使用 hw[0]；若表示 32-bit Thumb，slot0 使用连续的 hw[0] 和 hw[1]。
3. **确定 slot1 起点。** slot0 为 16-bit 时，下一条指令从 hw[1] 开始；slot0 为 32-bit 时，hw[1] 已属于 slot0，下一条指令必须从 hw[2] 开始。
4. **确定 slot1 长度。** 对 slot1 的起始半字再次执行相同的 Thumb 长度判断。slot1 可以独立为 16-bit 或 32-bit，不要求与 slot0 长度相同。
5. **检查完整性。** 16-bit 指令只需要起始半字有效；32-bit 指令必须保证起始半字和紧随其后的第二半字都有效。缺少第二半字时不能输出半条正常指令。

cm7pfu_fifo: slot0 与 slot1 的四种半字布局



图中从左到右是统一的逻辑读窗口 `hw[0]..hw[3]`，蓝色半字属于 slot0，绿色半字属于 slot1，灰色半字不属于本拍的两个输出。四行覆盖所有长度组合：

slot0 + slot1	slot0 使用	slot1 使用	两个 slot 都有效至少需要
16-bit + 16-bit	hw[0]	hw[1]	2 个有效半字
16-bit + 32-bit	hw[0]	hw[1] + hw[2]	3 个有效半字
32-bit + 16-bit	hw[0] + hw[1]	hw[2]	3 个有效半字
32-bit + 32-bit	hw[0] + hw[1]	hw[2] + hw[3]	4 个有效半字

例如，队首依次是 `A(16-bit)`、`B0/B1(32-bit)`、`C(16-bit)`。slot0 先取 A；由于 A 只占 `hw[0]`，slot1 从 `hw[1]` 开始。`hw[1]` 的编码说明 B 是 32-bit，因此 slot1 必须同时取得 `hw[1]=B0` 和 `hw[2]=B1`。`hw[3]=C` 不属于本拍两个 slot，只有 A、B 被 pop 后才成为新的 slot0。

半字数量不足时，输出按“能形成完整指令为止”停止：

当前连续有效半字	队首指令情况	slot0	slot1
只有 hw[0]	hw[0] 是 16-bit	valid	invalid
只有 hw[0]	hw[0] 是 32-bit 首半	invalid，等待 hw[1]	invalid
有 hw[0..1]	slot0 为 16-bit，hw[1] 也是 16-bit	valid	valid
有 hw[0..1]	slot0 为 16-bit，hw[1] 是 32-bit 首半	valid	invalid，等待 hw[2]

当前连续有效半字	队首指令情况	slot0	slot1
有 <code>hw[0..1]</code>	slot0 为 32-bit	valid	invalid，下一条应从尚未到达的 <code>hw[2]</code> 开始

因此，slot1 valid 必然蕴含 slot0 valid，接口不会越过一条不完整的老指令而输出更年轻的指令。DPU 也不能只消费 slot1: `pop_i=10` 是非法编码，消费两条必须使用 `pop_i=11`。

两个指令数据端口始终是固定的 32 bit，size 和 valid 用来说明其中哪些位有意义：

指令长度	32-bit 输出总线内容	DPU 应如何解释
16-bit	<code>[31:16]</code> 是当前指令半字； <code>[15:0]</code> 是读窗口中的下一半字或无效残留。	只使用 <code>[31:16]</code> ，低 16 bit 不属于该指令，不能译码或消费。
32-bit	<code>[31:16]</code> 是指令第一半字， <code>[15:0]</code> 是指令第二半字。	两个半字共同组成一条指令。

`i0_s_o/i1_s_o=1` 表示对应 slot 是 32-bit Thumb，0 表示 16-bit Thumb；只有相应 `i0_v_o/i1_v_o=1` 时，data 和 size 才能被下游采样。

每条指令的伴随属性按实际占用半字组合。16-bit 指令的 error 和 protection 只取起始半字；32-bit 指令任一半字错误都会令 instruction error 成立，而 protection 只有在两个半字都受保护时才成立。hit 输出保留两位，与数据总线半字顺序一致：bit1 对应第一半字，bit0 对应第二半字；16-bit 指令只使用第一半字的 hit。队尾错误导致第二半字永远不会到达时，slot0 的特殊强制推进规则见 5.2.7，不能把该异常规则用于普通数据不足场景。

5.2.5 pop、搬移与计数更新

先用输出 valid 屏蔽 DPU 请求，得到有效 pop: `popq = pop_i & {i1_v, i0_v}`。slot1 只有在 slot0 同时被消费时才允许 pop。有效 pop 对应的半字数如下：

DPU 有效 pop	slot0 大小	slot1 大小	消费 halfword 数
<code>00</code>	任意	任意	0
<code>01</code>	16-bit	不消费	1
<code>01</code>	32-bit	不消费	2
<code>11</code>	16-bit	16-bit	2
<code>11</code>	16-bit	32-bit	3
<code>11</code>	32-bit	16-bit	3
<code>11</code>	32-bit	32-bit	4

同一拍可能同时执行“DPU pop”和“imem 搬入主 FIFO”。计算顺序必须等价于以下算法：

1. 根据 slot size 生成最多四位的半字 pop mask。

2. 分别统计被消费半字中有多少当前位于主 FIFO、多少当前位于 `imem`。
3. 用 pop 后的主 FIFO 空间判断能否容纳 `imem` 的全部剩余有效半字。该实现只做整批搬入，不做部分搬入。
4. 若允许搬入，搬移起点为 `imem_rp + popped_from_imem`，因此被 DPU 直接从 `imem` 消费的半字不会再次写入主 FIFO。
5. 更新 `fifo_n = old_fifo_n - popped_from_fifo + moved_from_imem`；若发生搬入，`imem_n` 清 0，否则仅保留尚未被直接消费的语义。
6. 主 FIFO 读指针只按 `popped_from_fifo` 旋转，写指针只按 `moved_from_imem` 增加，二者均 modulo 12。

步骤 2 是跨边界场景正确计数的关键。例如一条 32-bit 指令的第一半在主 FIFO、第二半在 `imem`，pop 一条指令时，主 FIFO 读指针前进 1，`imem` 搬移起点也跳过 1；不能让任一物理指针前进 2。

5.2.6 full 与预取背压

物理容量是 16 个半字，但 `nxt_full_o` 并不等到 16 项才置位。它根据本拍 pop、搬移、push 后的下一总占用计算：

```
next_occupied = next_fifo_n + next_imem_n
nxt_full_o    = !flush_i && (next_occupied > 8)
```

主 FIFO 是否可接收 `imem` 也按 pop 后空间提前判断：无 pop 时要求 `fifo_n <= 8`；有 pop 时阈值增加本拍将从主 FIFO 消费的半字数。这样可以在同一上升沿释放旧项并写入最多四个新项，而不会保守地多停一拍。

`nxt_full_o` 是预取节流阈值，不是物理 overflow 标志。顶层把它寄存为 `fifo_full`，随后阻止新的顺序 fetch；8-halfword 阈值使已启动事务仍有机会使用 16-halfword 的物理余量。重实现必须保留“超过 8 即背压”和 16-halfword 的物理容量，不能用常见的 `count==depth` full 判定替换。full 期间最多再提交一块；该块提交后，在 full 解除或 flush 前不得继续用 `data_v_i` 覆盖输入缓冲。

`fifo_full` 不会令 PFU 进入 BLK。它只在 RUN 状态门控新的普通取指有效信号：`v_s_pf = run_pf && !fifo_full && bt_fe_ok`。因此 full 期间的行为是：状态机保持 RUN；不再发起新的顺序/BTAC 取指请求；FIFO 现有 slot 继续对 DPU 可见并允许 pop；已经进入 FE 的数据事务仍可利用预留的物理空间完成最后一次提交。DPU pop 降低占用后，`nxt_full_o` 撤销，PFU 无需经过状态切换即可继续生成普通请求。

`pfu_dpu_idle_o=1` 也不能用来推断状态为 BLK。该输出在 `RUN && (fifo_full || !bt_fe_ok)` 时同样成立，含义是 PFU 当前因数据或元数据背压而无需继续预取；只有 stop、pending stop 或同步错误等控制事件才进入 BLK。

5.2.7 错误尾部与 flush

`expt_n_i` 以半字数标记当前队列尾部的错误范围，合法值为 0..4。它只允许在此前发生 push 或 flush 后改变；错误有效期间不得再 push 正常 block。对逻辑窗口中第 `k` 个半字，错误条件等价于：该半字已落入“当前总占用的最后 `expt_n_i` 个半字”。32-bit 指令只要任一半字错误，slot error 就成立。

一个特殊情况是：队尾半字看起来是 32-bit 指令首半，但错误已使取指停止，第二半永远不会到达。普通 valid 规则会永久等待。`frc_e0` 因而强制 slot0 valid，让 DPU 接收错误；内部 pop 计算把它临时视作 16-bit，并清除 BTAC hit，确保异常能前进且不会用不完整指令更新预测器。对外 `size` 仍反映首半字解码结果，错误处理不能依赖缺失的第二半字。

`flush_i` 优先于同拍 push、pop 和搬移：下一拍 `imem_n=0`、`fifo_n=0`、`imem_rp=0`、`fifo_wp=0`、`fifo_rp=entry0`，并清除 `frc_e0`。数据寄存器本身可保留旧位模式，但所有 valid/count 都必须清零，所以旧路径内容不可再被观察为有效指令。

5.2.8 关键状态与重实现不变量

FIFO 的关键状态合同如下：

状态	reset	更新规则
imem_rp	0	新 block push 时取 data_a_i；imem 全部搬入主 FIFO后回 0；flush 回 0。
imem_n	0	push 时取饱和后的 data_e-data_a+1；搬入后清 0；flush 清 0。
fifo_wp	0	按本拍搬入 0..4 个 halfword 增加，modulo 12；flush 回 0。
fifo_rp	one-hot entry0	按实际消费且位于主 FIFO 的 halfword 数旋转；flush 回 entry0。
fifo_n	0	old + moved_from_imem - popped_from_fifo；flush 清 0。
nxt_full_o	0 的效果	fifo_n + imem_n 的下一值大于 8 时置位；该阈值给仍在途的 64-bit block 留出物理余量。

重实现至少应断言以下不变量：总有效半字数不超过 16；主 FIFO 有效数不超过 12；每拍主 FIFO push/pop 均不超过 4；fifo_rp 为 one-hot；slot1 valid 蕴含 slot0 valid；pop slot1 蕴含 pop slot0；逻辑 hw[k] 不得同时由主 FIFO 和 imem 驱动；非 flush push 时两个 32-bit word 均已到达。

5.3 Branch Prediction 与 BTAC

分支预测利用历史信息在分支真正执行前选择下一取指地址。BTAC entry 同时保存“这个位置是否曾是分支”“目标相对偏移”和“近期方向倾向”。它只改善前端时序；最终程序流仍由 DPU 执行结果决定。

PF阶段查询BTAC时只有取指地址，还没有取得并译码对应指令，因此不能先判断“这是分支”再决定是否查询。正确模型是：每个符合条件的取指块地址都并行查询BTAC，但只有此前由DPU识别并写入BTAC的分支地址才可能命中。普通指令地址同样参与地址比较，只是BTAC中没有它的有效entry，所以结果为miss并继续顺序取指。

必须区分BTAC的查询路径和维护路径：

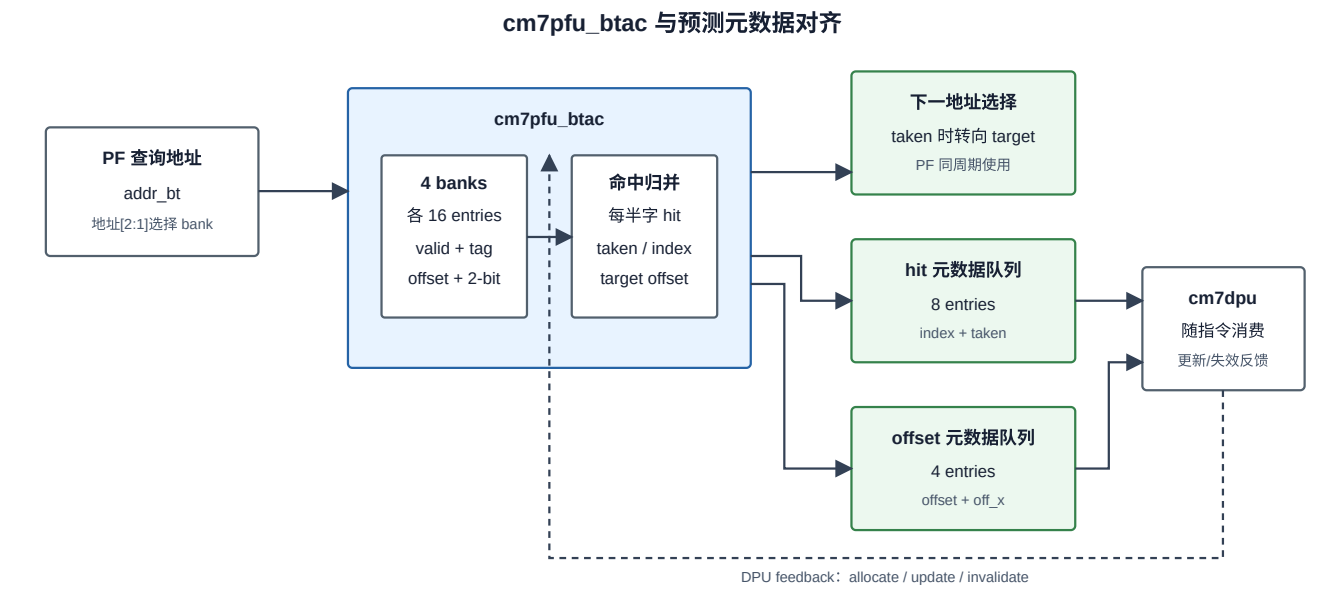
操作	发起者与时机	输入内容	结果
lookup	PFU对正常RUN取指、force pending恢复地址或当前DPU force地址执行。	当前64-bit取指块的地 址。	无匹配时顺序取指；分支 entry命中且预测taken时选择 预测目标。
allocate	DPU已经识别出可进入BTAC的分 支，但查询时没有可用entry。	分支实际地址、目标偏 移和实际方向。	在对应bank按round-robin位 置建立新entry。
update	DPU执行分支后得到实际方向， 或发现目标发生变化。	原entry index、实际 taken状态和必要的新 目标。	更新方向状态；full update还 更新tag和offset。
invalidate	DPU要求删除错误entry，或BTAC 检测到同bank multiple-hit。	查询地址及待失效 entry。	清除选中entry的valid，不影 响整张BTAC表。

一次lookup覆盖当前64-bit block中从请求起点开始的多个半字位置，而不是只比较block第一个地址。例如，取指令块为：

地址	内容
0x1000	普通指令 A
0x1002	普通指令 B
0x1004	分支指令 C
0x1006	后续半字

PFU用 0x1000 查询时，四个bank分别代表块内四个半字位置。A和B没有entry，因此不命中；若DPU此前已经为C的地址 0x1004 分配entry，则对应bank命中。PFU由此可以在64-bit指令数据尚未返回、C尚未再次进入DPU译码之前，提前取得C的方向和目标预测。若查询从 0x1002 开始，位于起点之前的 0x1000 不参与本次有效命中。

并非所有PFU状态都产生有效BTAC查询。RTL把普通RUN取指、FRP恢复和DPU force合并为 bt_lkp_v；reset/exception vector table读取不作为普通分支预测查询。BTAC hit功能关闭、FIFO full或side queue背压导致没有有效普通取指时，也不能产生有效预测命中。这里保留的RTL合同为 bt_lkp_v = v_s_pf || frp_pf || dpu_pfu_frc_v_i，其中表达式描述有效时机，而不是“这些地址已经确定是分支”。



cm7pfu_btac 固定为 4 banks × 16 entries。branch address [2:1] 选择起始 bank；从该 bank 到更高 bank 的比較器参与本次 64-bit fetch block lookup，使同一块内后续 halfword 的 branch 也能被发现。每 bank 有独立 4-bit wr_pt，allocation 写当前指针后加一，因此替换策略是“每 bank round-robin”，不使用 Least Recently Used (LRU，最近最少使用) 信息。

BTAC lookup接口接收完整的 btac_lkup_addr_i[31:1]，但当前RTL并不比较全部地址位。命中相关地址位被拆成 tag和bank两部分：

地址位	当前实现中的用途
[31:15]	输入端口携带这些位，但当前 hash_addr() 结果不使用它们，因此不参与tag命中比较。
[14:3]	形成12-bit lookup tag，与entry的 tag[11:0] 比较。
[2:1]	选择64-bit block中的起始bank；从该bank到bank3代表本次起点及其后的半字位置。
[0]	不进入BTAC地址接口；Thumb取指以16-bit半字对齐。

因此可以把当前实现概括为“地址 [14:1] 决定BTAC命中位置”，但不能把 [14:1] 误写成一个连续14-bit tag：其中 [14:3] 负责tag比较， [2:1] 负责bank选择。一个有效hit还必须同时满足以下条件：

- 1. 本拍lookup有效，且BTAC hit功能已使能。
- 2. entry的valid位为1。
- 3. entry tag等于查询地址 [14:3]。
- 4. entry位于查询起始bank或其后的bank，不能命中当前64-bit block起点之前的半字。
- 5. 当前lookup没有被显式invalidate或内部invalidate流程屏蔽。

由于 [31:15] 不参加比较，不同高地址但低 [14:1] 相同的分支会形成BTAC地址别名。例如 0x00001004 和 0x20001004 具有相同的 [14:1]，查询时会选择相同bank并比较相同tag。旧entry可能因此在另一个高地址上产生一次错误命中，甚至提供不适用于当前分支的方向或目标偏移。BTAC只是性能预测结构，这种别名不允许改变架构正确性：DPU执行真实指令后会检测方向或目标错误，通过force恢复正确地址，并update或invalidate对应entry。

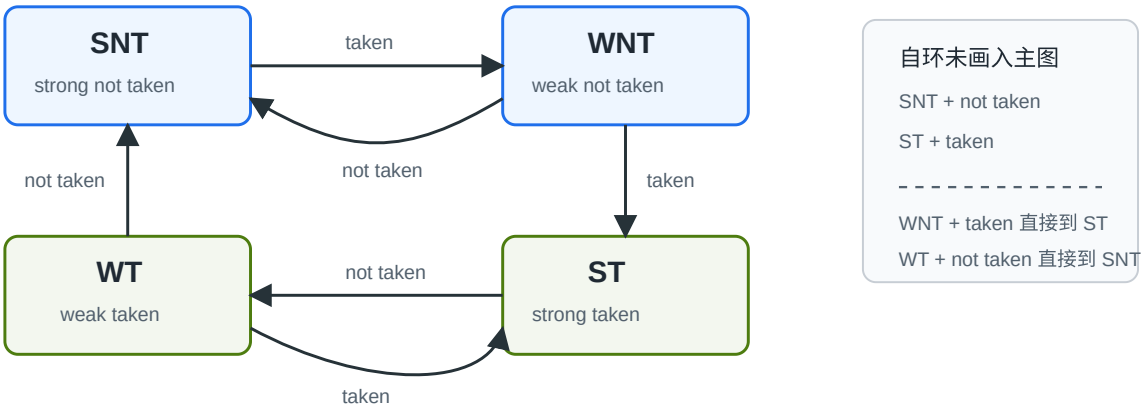
每个 27-bit entry 格式如下：

位	字段	语义
[26]	valid	reset 清 0；allocation 置 1；invalidate 清 0。
[25:14]	tag	12-bit部分地址tag；当前 hash_addr(addr) 实际返回 addr[14:3]。
[13:2]	offset	12-bit branch target 相对偏移。
[1:0]	direction	00 SNT、01 WNT、10 WT、11 ST；预测 taken 读取 bit1。

Strongly/Weakly Not Taken (SNT/WNT，强/弱不跳转) 与 Weakly/Strongly Taken (WT/ST，弱/强跳转) 的状态转移必须逐项复制下表，不能用常见的普通饱和和加减器替代：

当前状态	实际 not-taken	实际 taken
SNT 00	SNT 00	WNT 01
WNT 01	SNT 00	ST 11
WT 10	SNT 00	ST 11
ST 11	WT 10	ST 11

cm7pfu_btac taken predictor 状态跳转图



该状态图表达一次 DPU update 后的方向变化。allocation 不使用该转移：新 entry 在实际 taken 时初始化为 10，not-taken 时初始化为 01。lookup 同 bank 出现多个 tag hit 时，hit 信息仍保留用于后续维护，但 taken 被屏蔽；inv_line 选择该 bank 中最低 index 的命中 entry 清除 valid，留下的 entry 可继续服务后续 lookup。因此 multiple-hit 当拍不得改变下一取指地址，也不得一次无选择地清空整个 bank。

5.4 BTAC Side Queues

BTAC lookup 与 DPU 消费之间有可变数量的 FIFO 指令，因此预测元数据不能直接使用当前 lookup 输出。

队列	深度/吞吐	写入	读出	full 行为
bt_d	8 entries; 每拍最多写 4、读 1	每个有效 hit halfword 写 {index,taken}。	DPU pop 的指令包含 BTAC hit 时读 1。	根据下一剩余空间是否能容纳潜在 4 hit 提前产生 bt_f_pf，stall PF。
pc_d	4 entries; 每拍最多写 1、读 1	本 fetch 存在 taken hit 时写 {off_x, adjusted_offset}。	被 pop 的 hit 指令预测 taken 时读 1。	剩余空间不足以容纳下一 taken hit 时 pc_fl 阻止新 fetch。

flush 将 bt_wp/pc_wp 回 0，并将读指针恢复为空队列编码。side queue full 只暂停继续预取，不得丢弃已经排队的 metadata。

5.5 关键顶层状态与复位值

状态	复位值	可编码更新条件
state_pf	RST	st_adv = !stall_pf int_req force 时写 nxt_state。
addr_pf_q	RAR reset 下全 1	force 或 v_pf&&!stall_pf 时写 force/target/sequential。
addr_fe	31'h2	reset/interrupt 保存 vector table 地址；PF 推进保存 addr_pf。
v_fe	0	PF 成功推进时置位；FE 完成或 flush 清除；set/clear 同时发生时 set 值按 RTL 赋入。
priv	1	interrupt 置 1；否则 DPU privilege enable 时更新。

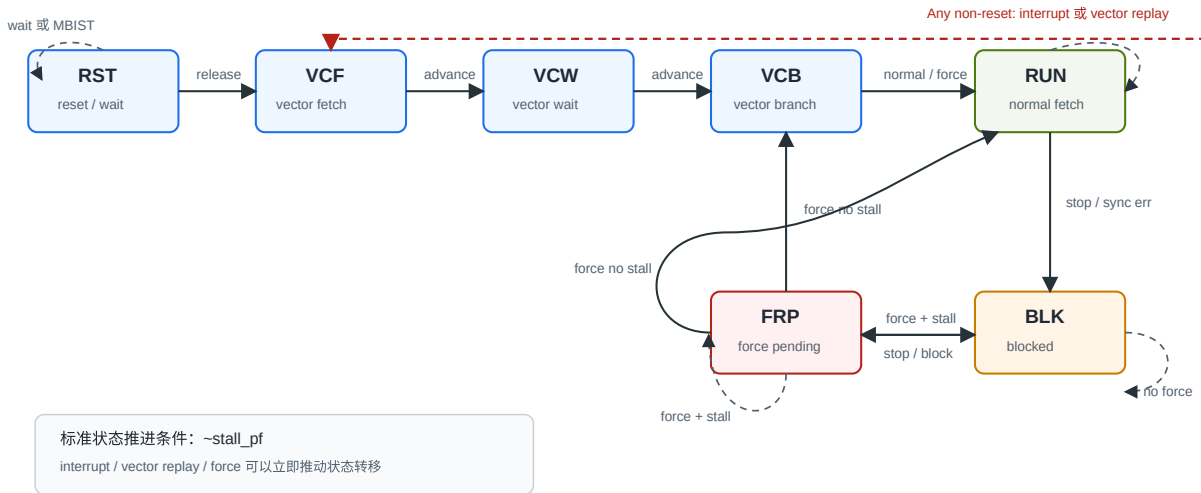
状态	复位值	可编码更新条件
<code>nmihf</code>	0	每拍采样 DPU NMI/HardFault 状态。
<code>rst_vc</code>	1	reset vector 正常交付后清 0；reset vector replay 保持 reset 语义。
<code>blp_v</code>	0	stop 在 stall 中置位；stall 解除或更高优先级 force 清除。
<code>ex_n_iss/ex_c_iss</code>	0 / RAR 值	FIFO push、replay、flush 按异常范围与优先级更新。

6. State Machines And Algorithms

6.1 PFU 主状态机

当前状态	事件/条件，按优先级	下一状态	动作
任意状态	<code>int_req = DPU interrupt vector replay</code>	<code>VCF</code>	最高优先级；flush旧路径并准备新的vector table地址。正常系统约束不在 <code>RST</code> 期间提出interrupt。
<code>RST</code>	<code>cpuwait MBIST</code>	<code>RST</code>	不发普通 fetch。
<code>RST</code>	<code>!cpuwait && !MBIST</code> 且允许state advance	<code>VCF</code>	离开复位等待，下一状态发起reset vector table取数。
<code>VCF</code>	address stall	<code>VCF</code>	保持vector table请求、地址和属性，等待ICU/TCU ack。
<code>VCF</code>	地址阶段可推进	<code>VCW</code>	vector table请求已被接受，转入数据等待。
<code>VCW</code>	data stall	<code>VCW</code>	保持FE事务，等待完整vector table数据或取消。
<code>VCW</code>	vector数据阶段完成	<code>VCB</code>	选择vector PC/MSP，并准备从vector PC取第一块指令。
<code>VCB/RUN/FRP/BLK</code>	<code>force && stall_pf</code>	<code>FRP</code>	force优先保存正确地址和待处理BTAC invalidate，但不能立即完成新地址事务。
<code>VCB/RUN/FRP/BLK</code>	<code>stop</code> 且没有更高优先级的 <code>force&&stall</code>	<code>BLK</code>	flush并停止继续预取；若stop遇到stall则先记入 <code>blp_v</code> 。
<code>VCB/RUN/FRP/BLK</code>	<code>force && !stall_pf</code>	<code>RUN</code>	flush旧路径，从force address恢复普通运行。
<code>VCB</code>	无force/stop/pending stop	<code>RUN</code>	向DPU交付vector PC；reset时同时交付MSP，并开始 handler/reset入口取指。
<code>RUN</code>	<code>blp_v expt_blk</code>	<code>BLK</code>	pending stop或同步取指异常阻止后续正常提交。
<code>RUN</code>	无退出事件	<code>RUN</code>	继续顺序/BTAC取指；FIFO或side queue满时可以留在RUN但不产生新普通请求。
<code>FRP</code>	原stall仍存在且无新异步事件	<code>FRP</code>	状态寄存器保持，保存的force地址不能被旧事务覆盖。
<code>FRP</code>	stall解除且无stop	<code>RUN</code>	使用保存的force地址完成非顺序恢复请求。
<code>BLK</code>	无interrupt/force	<code>BLK</code>	保持阻塞，取消/屏蔽旧response并报告 <code>idle=1</code> 。

cm7pfu 主状态机状态跳转图



图中的主干 `RST -> VCF -> VCW -> VCB -> RUN` 表示reset启动过程；interrupt和vector replay直接从当前状态转入 `VCF`，复用后三个vector状态。图中为可读性省略了多数stall自环：自环表示当前request、地址、目标选择和FE上下文保持，不是重新创建一笔独立事务。

6.1.1 全局推进、优先级与保持规则

状态寄存器复位为 RST。普通状态转移只在 `stall_pf=0` 时写入；`interrupt`或`force`属于必须及时处理的程序流事件，即使存在`stall`也能令状态寄存器更新。等价的状态写使能为 `st_adv = !stall_pf || int_req || force`。

int_req 由DPU interrupt或vector replay组成，优先于当前状态的所有普通分支，直接选择 VCF。在 VCB/RUN/FRP/BLK 内部，剩余优先级为：force&&stall 进入 FRP，然后是stop进入 BLK，再是无stall force进入 RUN，最后才考虑pending stop、同步错误或正常路径。这个优先级意味着stop和force同拍且PF未stall时，stop优先；force和stop同拍但PF正在stall时，force pending优先。

stop本身不在异步state write enable中。若stop到达时PF正在stall，b1p_v先记住该请求，状态和旧事务保持；stall解除后，b1p_v再令状态进入BLK。因此，重实现不能把单拍stop在stall期间直接丢失，也不能在没有取消旧事务前提前进入可接受新数据的状态。

6.1.2 RST：复位等待与vector地址准备

RST是硬件复位后的唯一入口，`pfu_in_reset_o=1`。该状态不产生普通RUN取指，也不向FIFO提交指令；
`rst_vc=1` 标记当前尚未完成reset vector流程。PFU在此阶段根据Vector Table Offset Register (VTOR，向量表偏移寄存器)准备reset vector table地址，reset地址选择vector table的初始MSP和reset PC所在64-bit block。

`cpuwait_i=1` 表示系统要求CPU在复位后继续等待，`miu_prod_mbist_en_i=1` 表示存储阵列仍在执行MBIST；任一条件成立都保持RST。两者均解除后，状态进入VCF。RST的退出不是已经取到reset PC，而只是允许下一状态开始读取vector table。

6.1.3 VCF：发起vector table取数

VCF 负责vector table fetch的地址阶段。reset路径使用VTOR基地址处包含初始MSP和reset PC的block；普通exception使用VTOR基地址加ISR number形成的vector entry地址。该请求标记为非顺序和 first 访问，并通过 pfu_icu_vf_o/pfu_tcu_vf_o 以及MPU vector-fetch属性告诉下游这是vector读取，而不是普通指令FIFO填充。

PFU仍按地址选择 `cm7icu` 或 `cm7tcu`。如果请求尚未ack, `a_stall` 使状态保持 `VCF`, 请求地址、privilege、target 和vector属性必须保持稳定; 不能每拍重复生成新的vector事务。地址阶段被接受且PF可推进后进入 `vcw`。

6.1.4 VCW：等待vector数据

VCW 表示vector地址已经被存储系统接受，PFU正在等待数据阶段。该状态不发下一笔普通取指，FE保存vector事务的地址和ICU/TCU选择，直到ICU dvalid 或TCU data_complete 到达。返回的64-bit数据写入 cm7pfu_fifo 的 memory-data寄存器供vector选择逻辑读取，但不会按普通指令执行 fifo_push ，因为普通FIFO提交只允许在 RUN 状态发生。

等待期间 d_stall 传入 stall_pf ，状态保持 VCW 。vector bus error在该阶段按reset、NMI/HardFault或普通 exception上下文记录为reset vector fault、lockup或vector fault。数据完成或被错误/取消路径处理后进入 VCB 。

6.1.5 VCB：交付vector并启动入口取指

VCB 把上一状态取得的32-bit vector PC交给DPU， pfu_dpu_vect_v_o 在vector数据已登记时有效。reset流程还从同一64-bit block的另一word输出初始MSP，并拉起 pfu_dpu_msp_v_o ；普通exception只输出handler PC。大小端配置在交付前对选中的32-bit word执行byte重排。

同一状态还把vector PC作为非顺序、 first 取指地址，开始请求reset入口或exception handler的第一块指令。因此 VCB不是单纯的“通知DPU”状态，它同时建立从vector table数据到新指令流的桥接。目标地址请求stall时，VCB保持地址事务和关联上下文；vector valid由 VCB && fc_in_fe 限定，不能把VCB因stall多停的周期解释成重复交付多个 vector。正常完成后进入 RUN 。stop使其进入 BLK ，force可直接改用force地址，force遇到stall则转 FRP 。vector retry/ECC replay通过全局 int_req 重新进入 VCF ，不能把错误vector当作有效入口。

6.1.6 RUN：正常预取与FIFO提交

RUN 是程序正常执行期间的稳态。只有在FIFO未达到节流阈值、BTAC side queues仍有容量且没有外部/内部stall 时，PFU才生成新的普通取指请求。PF stage在顺序地址、BTAC预测目标以及可能的程序流覆盖之间选择地址；FE接收完整响应并在未取消、无pending同步错误时产生 fifo_push 。

留在 RUN 不等于每拍都必须发请求。FIFO full或BTAC元数据队列背压时，状态仍是 RUN ，但 v_s_pf=0 ，PFU暂停预取并可向DPU报告idle；DPU继续pop后即可恢复。普通BTAC taken命中只改变下一地址，不产生flush，也不离开 RUN 。

RUN中的条件	状态变化	新普通请求	现有FIFO输出	恢复方式
fifo_full=1	保持 RUN	停止	继续valid并允许DPU pop	pop降低占用后自动恢复。
bt_fe_ok=0，BTAC side queue背压	保持 RUN	停止	继续valid并允许DPU pop	元数据被消费、queue有空间后自动恢复。
stop 或 blp_v	转 BLK	停止	PFU flush后旧slot invalid	等待force或interrupt。
expt_blk=1	转 BLK	停止	保留可交付的错误语义，不再填充后续正常块	DPU处理异常后以 force/interrupt恢复。

前两行是可自动解除的流量控制，不清空FIFO，也不改变主状态；后两行是程序流/错误控制，会进入 BLK 。重实现不能为了复用“暂停”状态而把FIFO full映射为 BLK ，否则DPU pop后PFU无法按原行为自动恢复，并可能错误触发 cancel/flush。

interrupt/vector replay无条件转 VCF ；DPU force清除旧路径并从正确地址继续，stall时先进入 FRP ；stop或已锁存的 blp_v 进入 BLK ； expt_blk 表示FIFO已有同步错误范围，PFU停止继续填充并进入 BLK 等待DPU处理。

6.1.7 FRP：保存并重试force地址

Force Pending (FRP，强制地址等待)只在DPU给出force但PF仍被旧地址/数据事务stall时使用。force到达当拍已经触发flush和cancel，并把正确force地址写入地址保持状态；相关BTAC invalidate请求也被保存。进入FRP的目的不是继续旧路径，而是防止正确恢复地址在旧事务真正解除前丢失。

只要原stall仍存在且没有更高优先级事件，状态寄存器保持FRP，旧response不得提交。stall解除后，FRP把保存的force地址作为非顺序请求，并回到RUN。若等待期间收到stop则进入BLK；新的force可以覆盖恢复目标；interrupt/vector replay仍以最高优先级转入VCF。

6.1.8 BLK：停止预取并等待恢复事件

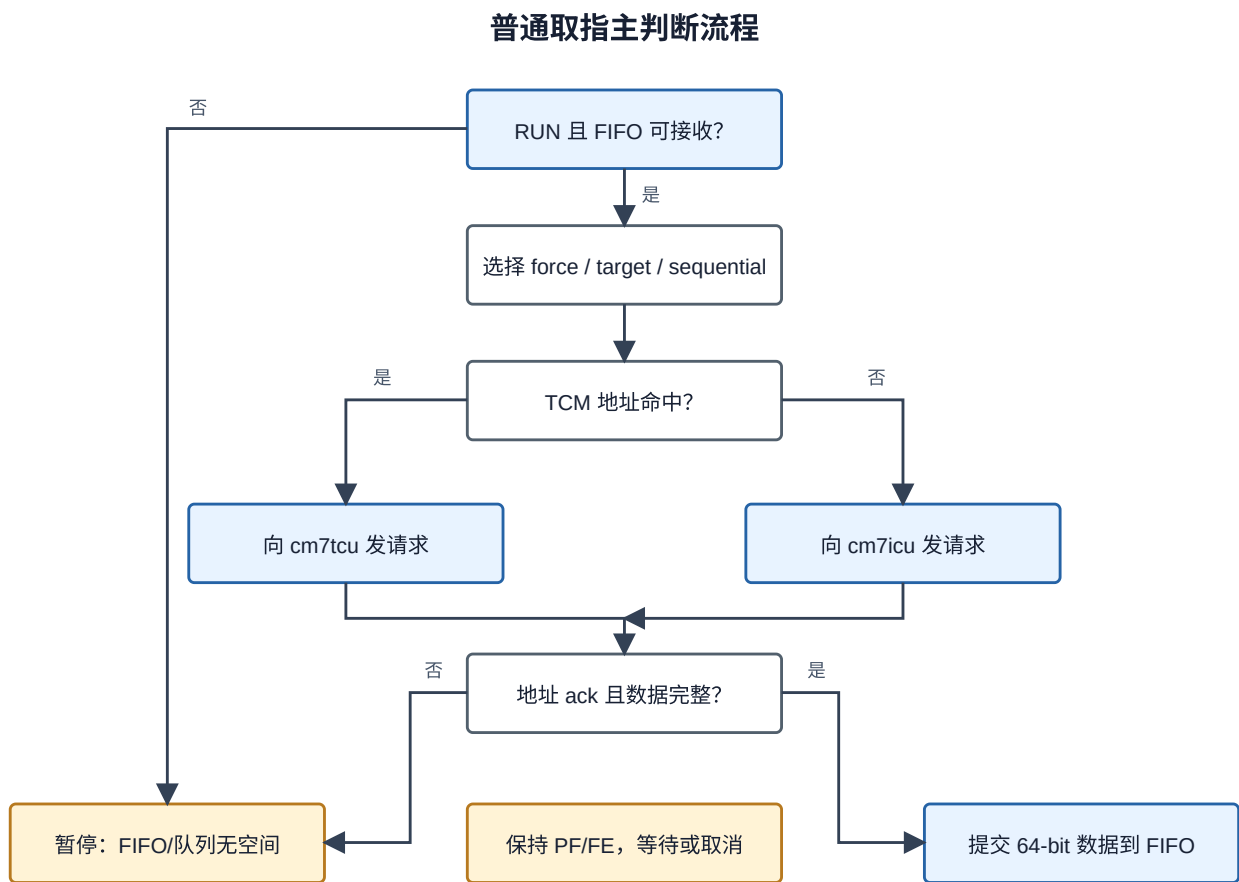
Blocked (BLK，阻塞)用于DPU stop、已锁存stop以及同步取指错误。该状态不产生普通或非顺序取指请求，blk_pf屏蔽ICU/TCU data-valid写入并令address/data cancel保持有效，防止迟到的旧response污染FIFO；pflu_dpu_idle_o=1表示PFU已停止预取。

BLK不会因为FIFO变空或存储器恢复就自动退出。无interrupt/force时始终保持；DPU必须在处理完同步错误或改变程序流后给出force，PFU才从指定地址回到RUN，若force仍受stall阻挡则先进入FRP。interrupt或vector replay直接转VCF。stop持续有效时保持BLK及flush/cancel语义，但不产生新的取指事务。

6.1.9 非法状态

编码3'b111没有恢复分支，组合next-state为未知值。兼容实现和验证环境必须把非法状态视为fatal设计错误，不能擅自跳回RST或RUN，否则会掩盖状态损坏并产生不可预测的存储事务。

6.2 普通取指算法



interrupt / force / stop / 同步异常可在任一周期边界触发 flush/cancel, 终止旧路径

可编码算法如下：

1. 仅当 `RUN`、FIFO 未达到节流阈值且两个 BTAC side queue 可接收时，生成普通顺序事务；vector/force/FRP 属于非顺序事务，可独立产生 `v_ns_pf`。
2. 用第 5.1 节优先级形成 `addr_pf`，并执行 TCM decode。ITCM 命中条件为 enable、地址顶 nibble `0x0`、size 非零且 mask 命中；DTCM 对应顶 nibble `0x2`。
3. 任一 TCM 命中发 TCU request，否则发 ICU request；同一逻辑事务只能选择一个 target。
4. ack 未到时 `a_stall=1`。地址成功推进后设置 `v_fe` 并保存 `addr_fe/cs_fe`。
5. FE 等待 ICU dvalid 或 TCU data_complete。等待期间 `d_stall=1`；出现 flush/cancel 时允许解除 FE stall，但响应不得 commit。
6. 正常完整响应满足 `run_pf && v_fe && !d_stall && (!cancel_fe || fake_dv) && !msk_push` 时产生 `fifo_push`。
7. MPU abort 使用全零 fake data 推动错误进入 FIFO，但同时 cancel 真实 memory response，防止 ICU 分配或旧数据污染。

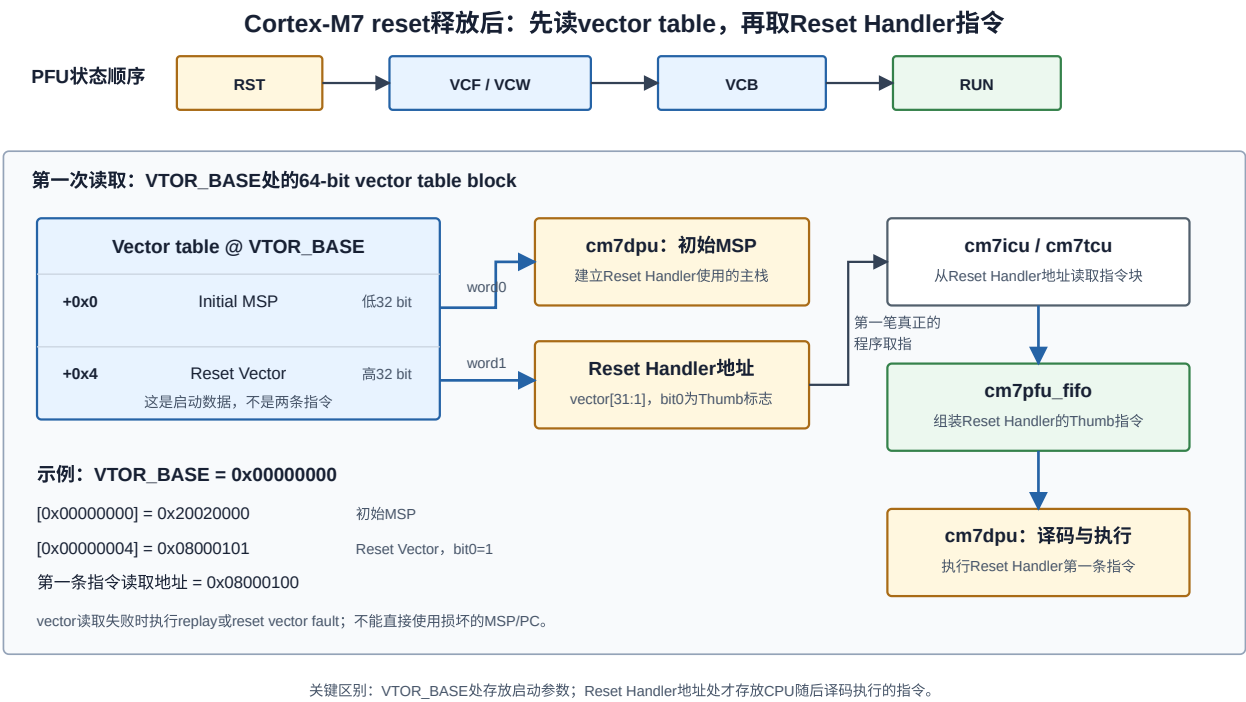
6.3 Armv7E-M Reset释放后的首次读取

Cortex-M7实现的是Armv7E-M架构。它的reset启动方式与“复位后直接从固定地址执行第一条指令”的CPU不同：reset释放后，处理器先把内存中的vector table当作启动描述数据读取，从中取得初始栈指针和Reset Handler入口地址，然后才去Reset Handler地址取第一条真正执行的指令。

Vector Table Offset Register (VTOR，向量表偏移寄存器)给出vector table基地址。本实现的VTOR复位值可由顶层INITVTOR[31:7]配置，PFU看到的是ppb_vto_tbloff_i[31:7]；常见配置为0x00000000，但重实现不能把基地址硬编码为0。基地址低7位固定为0，因此vector table至少按128 byte边界对齐。

reset时最先读取的两个32-bit word如下：

地址	内容	处理器用途
VTOR_BASE + 0x0	Initial Main Stack Pointer，初始MSP值。	装载Main Stack Pointer，建立Reset Handler开始执行时使用的主栈。
VTOR_BASE + 0x4	Reset Vector，即Reset Handler入口指针。	装载初始PC；随后从该指针表示的代码地址取指。



图中第一条蓝色数据流是vector table读取，它取得的是两个启动参数，不是两条指令。本RTL的数据接口宽度为64 bit，因此cm7pfu在一次对齐block中同时取得低word的初始MSP和高word的Reset Vector。该数据写入cm7pfu_fifo的memory-data寄存器供vector选择逻辑读取，但不会作为普通Thumb指令执行fifo_push。

第二条数据流才是第一笔真正的程序取指。PFU把Reset Vector的地址部分作为非顺序、first请求发送给cm7icu或cm7tcu；返回的Reset Handler指令块进入cm7pfu_fifo，组装后交给cm7dpu译码执行。Reset Vector最低位用于表示Thumb入口，正确的软件镜像应将bit0置1；PFU取指地址接口保存[31:1]，因此实际指令地址按半字边界使用Reset Vector的地址部分。

例如VTOR基地址为0x00000000，vector table内容为：

0x00000000	: 0x20020000	初始MSP
0x00000004	: 0x08000101	Reset Vector, bit0=1

处理器不会把 0x20020000 或 0x08000101 当作位于vector table中的指令执行，而是执行以下动作：

1. 将初始MSP设置为 0x20020000。
2. 从Reset Vector得到Reset Handler代码地址 0x08000100，最低Thumb标志位不进入半字对齐的存储器取指地址。
3. 向 0x08000100 发起第一笔普通指令读取。
4. 将返回的16-bit/32-bit Thumb指令经PFU FIFO送入DPU，开始执行Reset Handler。

该过程映射到PFU状态机如下：

状态	reset启动中的具体动作	交付结果
RST	等待 <code>cpuwait_i</code> 和MBIST解除，准备当前VTOR基地址。	尚未读取vector table，也没有普通指令。
VCF	向VTOR基地址发起vector table 64-bit读取。	请求低word初始MSP和高word Reset Vector。
VCW	等待ICU/TCU返回完整数据。	vector数据进入memory-data寄存器，不进入普通指令FIFO有效计数。
VCB	向DPU交付MSP和Reset Vector，并以Reset Handler地址发起非顺序 <code>first</code> 取指。	建立初始栈和初始PC，启动第一块程序指令读取。
RUN	接收Reset Handler指令块并正常push到PFU FIFO。	DPU取得并执行Reset Handler的第一条指令。

若vector table读取收到TCU retry或ICU ECC error，PFU执行vector replay并重新进入 `VCF`，不能使用可能损坏的MSP/PC。若发生vector bus error，则产生reset vector fault语义；它不是普通指令FIFO中的bus-error指令。`cpuwait_i` 或MBIST只延迟离开 `RST`，不会改变vector table中首先读取 `+0x0/+0x4` 两个word的顺序和含义。

6.4 通用Exception Vector Fetch算法

普通interrupt/exception的入口地址同样来自内存中的vector table，不由PFU或Nested Vectored Interrupt Controller (NVIC，嵌套向量中断控制器)硬编码。这里必须区分两个不同的地址：

名称	如何得到	含义
vector entry address	PFU使用当前 <code>VTOR_BASE</code> 和NVIC提供的exception number计算。	vector table中保存某个Handler指针的32-bit word地址。
Handler address	PFU从上述vector entry读取出的32-bit值。	exception handler代码入口；PFU随后从该地址开始取指。

NVIC只向PFU提供 `nvic_int_nxt_isr_i[7:0]` 异常编号N。每个vector entry占4 byte，因此架构地址关系为：

$$\text{vector_entry_address} = \text{VTOR_BASE} + 4 * N$$

当前RTL在 [31:1] 半字地址域中用 $\text{VTOR_BASE} \mid (N \ll 2)$ 的等价拼接形成 `isr_a`。在vector table满足实现所需合法对齐时，OR与上述加法结果相同；软件不能把未正确对齐的VTOR依赖为正常行为。典型vector位置如下：

内容/异常	Vector编号N	vector entry地址
Initial MSP	0	$\text{VTOR_BASE} + 0x00$
Reset	1	$\text{VTOR_BASE} + 0x04$ ；reset流程特殊地从基地址一次读取前两个word。
NMI	2	$\text{VTOR_BASE} + 0x08$
HardFault	3	$\text{VTOR_BASE} + 0x0C$
External IRQ0	16	$\text{VTOR_BASE} + 0x40$
External IRQk	16+k	$\text{VTOR_BASE} + 4 * (16+k)$

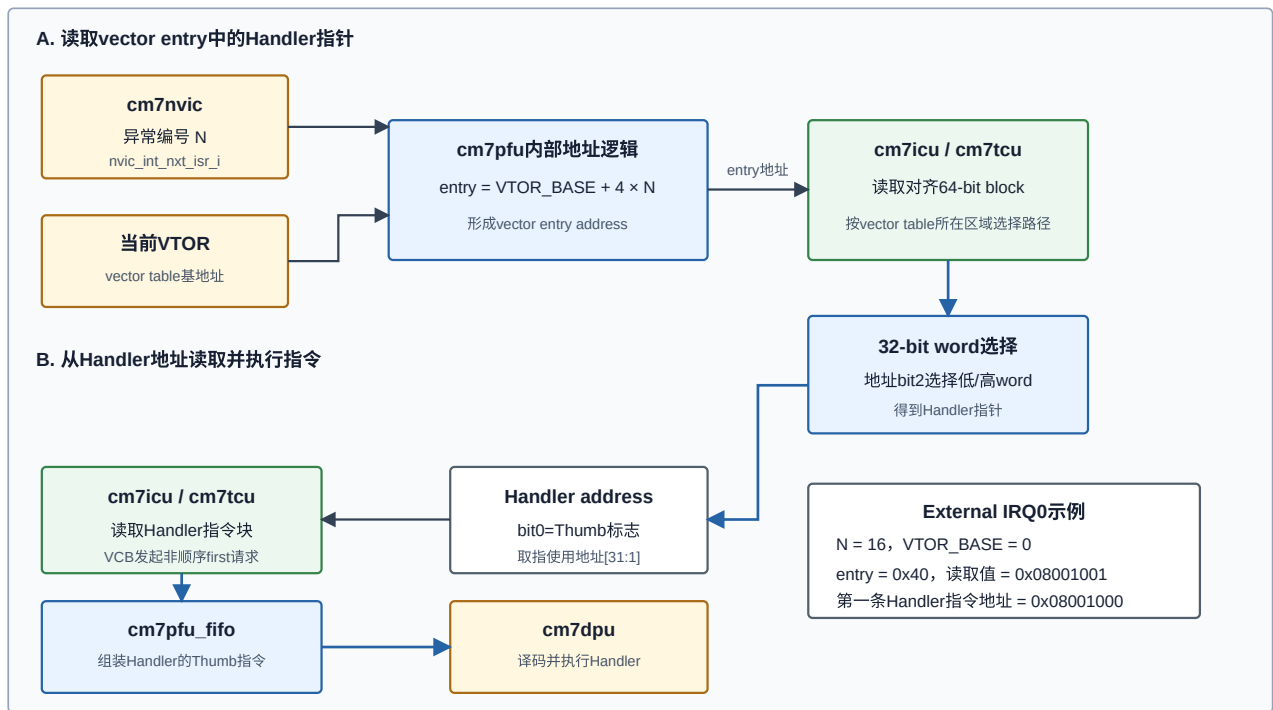
External IRQ0中的“0”是外部中断编号，不是vector table编号。Armv7E-M先保留vector index 0..15给Initial MSP和处理器内部异常，因此IRQ0对应vector index 16。每个entry为4 byte，所以其相对VTOR的字节偏移为 $16 * 4 = 64 = 0x40$ 。这里的 `0x40` 是地址偏移，不表示“第40个entry”。

`VTOR_BASE+0x40` 处保存的是一个32-bit Handler入口指针，不是IRQ0 Handler的机器指令。PFU需要先读取这个word，再用其中的地址部分发起第二次存储器访问，后者才返回Handler代码。例如：

```
VTOR_BASE           = 0x20000000
IRQ0 vector entry address = 0x20000040
[0x20000040]        = 0x00001001    // Handler指针
Handler instruction addr = 0x00001000    // 真正存放指令的位置
```

指针最低bit0为Thumb入口标志，因此vector table中通常保存 `Handler代码地址+1`；PFU/DPU保留该状态语义，而半字对齐的取指地址使用指针 [31:1]。Vector entry与Handler代码可以位于不同存储区域：上述 `0x20000040` 可能位于DTCM或普通RAM，而 `0x00001000` 可能位于ITCM；PFU分别对两个地址执行decode并选择对应读取路径。

cm7pfu：从异常编号取得Handler入口并开始取指



NVIC提供编号，vector table提供Handler地址；PFU负责地址计算、读取、word选择和后续指令取回。

图中上半部分是Handler指针读取：cm7nVIC 提供异常编号，cm7pfu 内部地址逻辑与VTOR基地址组合出vector entry address，再根据地址映射选择 cm7ICU 或 cm7TCU 读取。下半部分是程序取指：PFU从64-bit返回块中选出目标 32-bit word，把其中的Handler address交给DPU，并以该地址发起handler第一块指令请求。

存储器接口返回对齐的64-bit block，而一个vector entry只有32 bit，因此PFU用vector entry地址的bit2选择目标 word：

vector_entry_address[2]	选择内容
0	64-bit block低32-bit word。
1	64-bit block高32-bit word。

reset是特例：无论普通地址选择如何，reset流程都从VTOR基地址的64-bit block中取低word作为Initial MSP、高 word作为Reset Vector。普通interrupt/exception只选择自己的Handler word，不会像reset那样再次从vector table加载Initial MSP；exception entry期间使用哪个栈以及寄存器压栈由DPU的异常机制处理，不属于PFU vector读取职责。

例如 VTOR_BASE=0x00000000，NVIC选择External IRQ0，N=16：

```
vector entry address = 0x00000000 + 16 * 4 = 0x00000040
[0x00000040]         = 0x08001001
Handler取指地址      = 0x08001000
```

PFU先读取包含 0x00000040 的64-bit block并取得 0x08001001。最低bit0是Thumb入口标志，半字对齐的存储器取指地址使用 [31:1]，所以随后从 0x08001000 读取handler指令。vector table可以位于ITCM、DTCM或普通存储区域：命中ITCM/DTCM窗口时由 cm7TCU 读取，否则由 cm7ICU 读取。

bigend_i=1 时，PFU对选中的32-bit Handler word执行byte顺序反转后再交付。vector PC只在 VCB && fc_in_fe 时有效。若收到TCU retry或ICU ECC error， vect_replay 重新触发 VCF，不得把该word当成Handler地址；vector bus error则按NMI/HardFault lockup或普通vector fault语义处理。

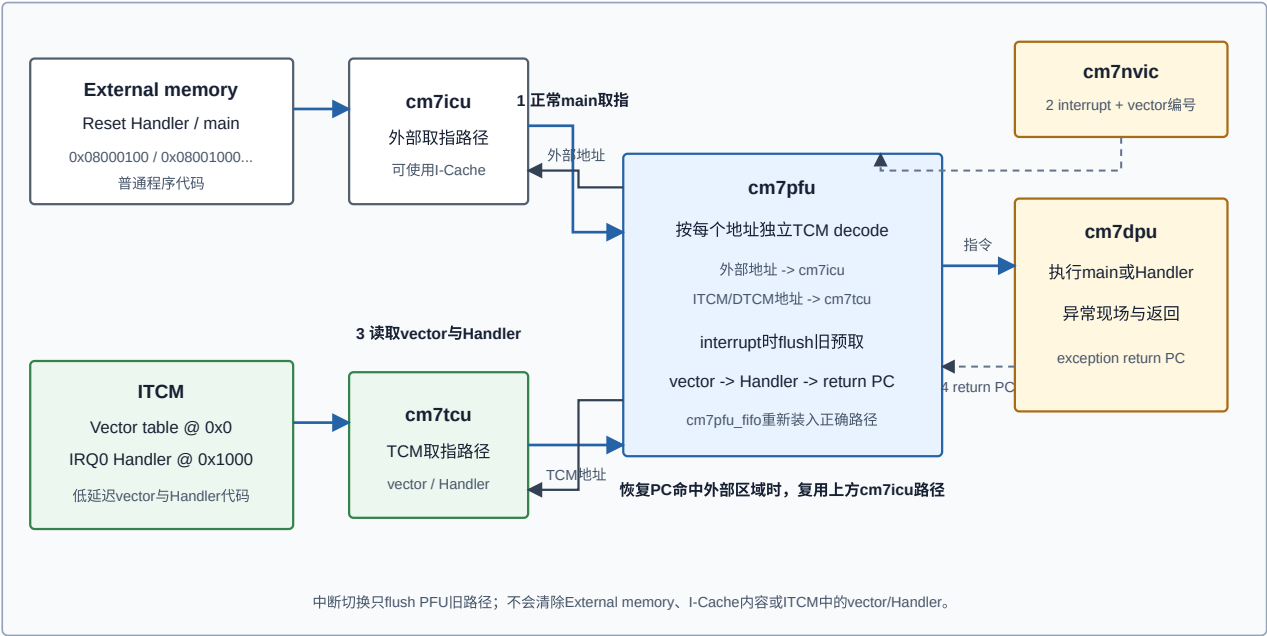
6.4.1 Vector table、Handler与主程序分区部署

“中断放在TCM”需要进一步区分vector table和Handler代码。PFU对vector entry读取地址、Handler取指地址以及异常返回后的主程序地址分别执行TCM decode，因此三者不要求位于同一种存储器：

Vector table位置	Handler代码位置	Vector entry读取路径	Handler指令读取路径
TCM	TCM	cm7tcu	cm7tcu
外部memory	TCM	cm7icu	cm7tcu
TCM	外部memory	cm7tcu	cm7icu
外部memory	外部memory	cm7icu	cm7icu

主程序可以始终位于外部Flash/memory并通过 cm7icu 取指，而vector table和关键中断Handler放在ITCM中以缩短中断入口取数延迟。当前RTL也能把指令请求decode到DTCM，但常规软件布局优先使用ITCM保存可执行Handler；无论选择哪种TCM，都必须与实际enable、size和MPU执行权限一致。

外部主程序与TCM Vector/Handler的混合取指路径



示例路径：main经ICU执行 -> interrupt -> vector与Handler经TCU执行 -> exception return -> main重新经ICU执行。

图中正常阶段的主程序地址命中外部memory路径，指令经 cm7icu 进入PFU。interrupt到达后，PFU flush主程序的预取FIFO并进入vector流程；VTOR指向ITCM，所以vector entry经 cm7tcu 读取。entry中保存的Handler地址也位于ITCM，随后Handler指令继续经 cm7tcu 进入PFU和DPU。exception return恢复的PC重新落入外部memory范围，PFU自动切回 cm7icu，不需要软件为每次取指显式选择ICU或TCU。

一种可用的地址布局示例如下：

```

ITCM vector table : 0x00000000
ITCM IRQ0 Handler : 0x00001000
External Reset code: 0x08000100
External main code : 0x08001000...

[0x00000000] = 0x20020000    Initial MSP
[0x00000004] = 0x08000101    Reset Handler位于外部memory
[0x00000040] = 0x00001001    External IRQ0 Handler位于ITCM
VTOR          = 0x00000000

```

在该布局中，reset vector table读取走TCU，Reset Handler和主程序取指走ICU；IRQ0发生后，vector entry和Handler取指都走TCU；异常返回后再次走ICU。PFU按每个地址独立选择目标，不会因为中断前正在使用ICU就强制Handler也走ICU。

实现和软件部署必须满足以下前提：

1. 对应 `itcm_en_i/dtcm_en_i` 已经使能，size配置覆盖vector table和Handler地址。
2. MPU属性允许从目标TCM区域执行指令；否则Handler取指会产生MPU abort。
3. VTOR满足vector table对齐要求，并指向实际有效的table副本。
4. 每个Handler entry保存的地址bit0为1，表示Thumb入口。
5. 链接脚本把vector table和指定Handler放入目标TCM section，并保证运行地址与entry值一致。
6. 若TCM复位后没有预装内容，启动代码必须先从外部memory复制vector table和Handler到TCM，确认内容可用后再写VTOR。修改VTOR之前到来的interrupt仍会使用旧vector table。
7. 若顶层 `INITVTOR` 在reset时就指向TCM，则TCM中的Initial MSP和Reset Vector必须在CPU离开 `RST` 前已经有效；不能依赖尚未执行的Reset Handler去填充自己的启动vector。

interrupt引起的PFU flush只清除旧主程序的预取状态和未完成事务，不清除ICU Cache、TCM内容或外部memory。DPU保留架构要求的异常现场；Handler完成后由exception return提供恢复PC，PFU再依据该PC重新选择外部memory或TCM路径。

6.5 Flush、Stall 与 Cancel

控制	精确组合条件	必须产生的效果
<code>flush</code>	interrupt/replay、DPU stop、DPU force、或非顺序状态在有效 edge 前进。	清 FIFO 和 side queues；清/覆盖 FE valid；阻止 stale instruction 对 DPU 可见。BTAC 正常 taken hit本身不 flush。
<code>stall_fe</code>	<code>v_fe && !ready_fe && !cancel_fe</code> ，再与 <code>!flush</code> 合成。	保持 FE transaction；不允许下一请求覆盖其数据上下文。
<code>stall_pf</code>	chip-select mispredict、BTAC queue capacity stall、address 未 ack、或 FE stall。	保持 PF 事务和状态机标准转移；interrupt/force 仍可改变控制状态。
data cancel	<code>flush ms_cd BLK</code> ，再加 target conflict 或 fake data。	ICU/TCU 后续 response 被屏蔽。

控制	精确组合条件	必须产生的效果
TCU address cancel	data cancel、ms_ca、target conflict、TCU/ICU overlap。	取消最后一个尚未安全完成的 TCU address。

chip-select mispredict 指 speculative TCM/ICU 选择与真实 tcm_dec(addr_pf) 不一致。若错误请求尚未 ack，记录 address cancel；若已 ack 且数据尚未完成，记录 data cancel。跨 target pipeline conflict 也必须取消新地址，不能仅依赖 cs_fe 在返回时重新选择。

本节表中的 flush 是 PFU 边界内的完整清空。DPU 内部不是由该信号直接复位所有 stages，而是由 flush_iss、br_kill_iss/ex1/ex2/wr 和 per-slot quash 完成按年龄失效。兼容实现必须同时具备这两层动作：只做 PFU flush 会让已经进入 DPU 的 wrong-path 指令继续执行；无差别 reset DPU 又会错误丢弃 branch 和 older instructions。

6.6 Error And Replay

错误先按有效 halfword 从低地址到高地址选择第一个，再使用以下编码。高位 bit3 表示 vector-context 标记。

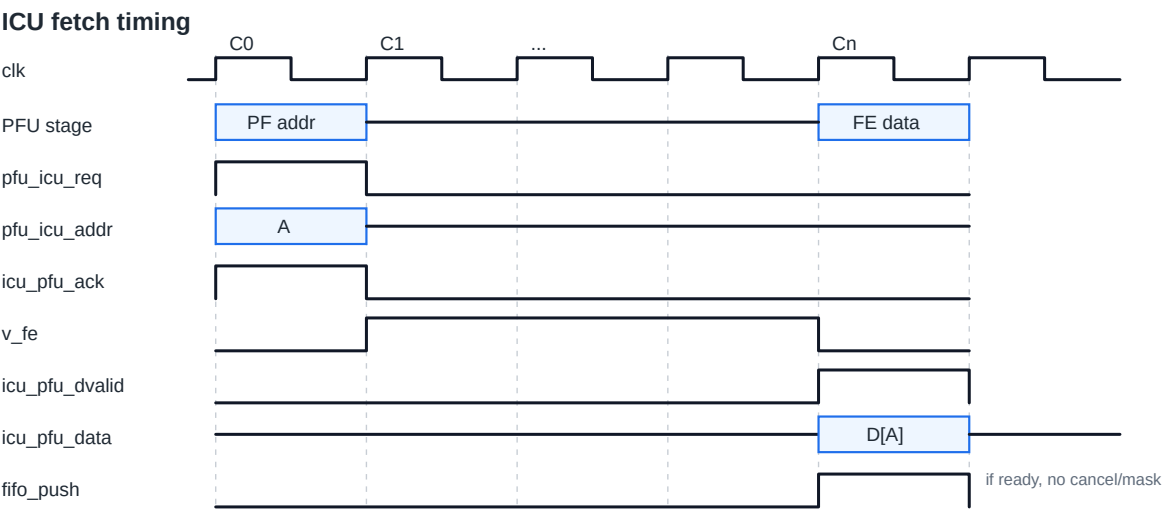
pfu_dpu_err_code_o	含义	触发
0001	breakpoint	FPB hit 且未被 debug protection 屏蔽。
0010	MPU abort	P1 abort；使用 fake data。
0011	bus error	ICU bus error 或 TCU error。
0101	replay	已 push 的有效块收到 TCU retry 或 ICU ECC error，且前面没有覆盖该范围的错误。
1100	vector catch	debug vector catch。
x110	vector lockup	NMI/HardFault vector bus error；bit3 是正交的 vector-catch 标记。
x111	vector fault	普通 exception vector bus error；bit3 是正交的 vector-catch 标记。
x000	reset vector fault	reset vector bus error；主要错误状态同时由 pfu_dpu_vect_e_o 表示。

优先关系为 vector lockup/fault/reset fault，随后 vector catch、breakpoint、MPU abort、bus error；replay 只在此前已提交范围没有更高优先级错误时成立。异常范围以 halfword count 保存在 ex_n_iss，msk_push 在 pending exception 或 replay 时阻止后续正常块进入 FIFO。RUN 看到 expt_blk 后进入 BLK，等待 DPU force/interrupt 恢复程序流。

7. Timing Diagrams

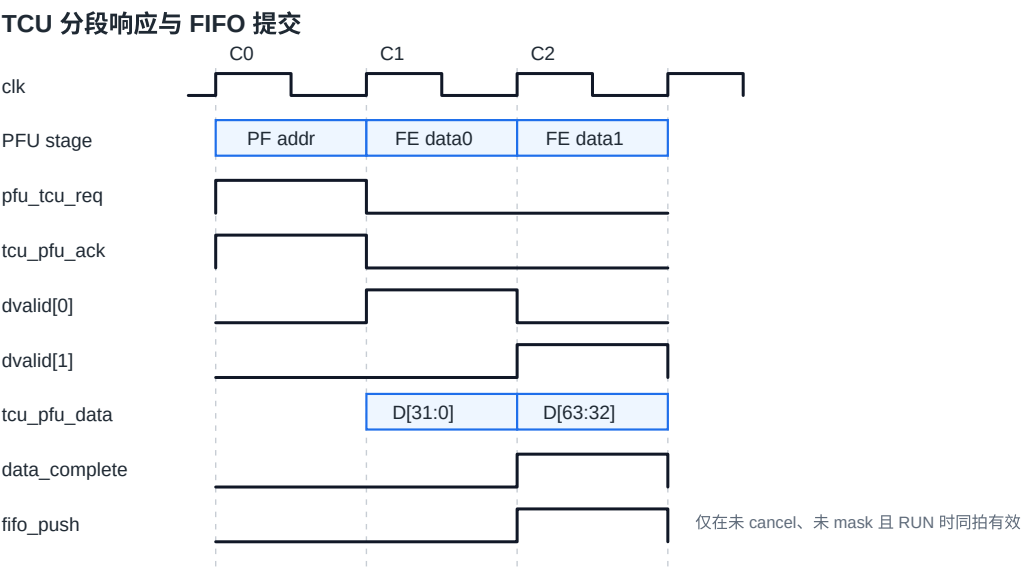
所有波形均以 clk 上升沿为采样边界，竖向 cycle grid 即允许同步信号变化的位置。图中等待长度是示例；协议允许可变延迟。

7.1 ICU Request / Data / FIFO Commit



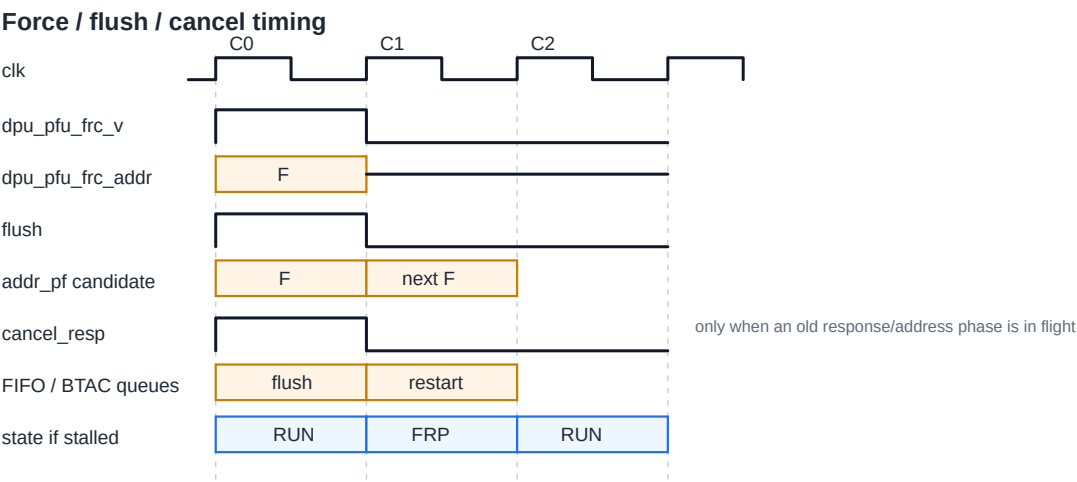
PFU 在 cycle 边界驱动 request/address。ack 未采样为 1 时，PF 保持事务并拉起 address stall；ack 被采样后，事务上下文进入 FE。ICU dvalid 可以在后续任意 cycle 到达，`icu_pfu_data_i` 与 dvalid 同周期有效。只有 dvalid、未 cancel、RUN、无 pending error 同时满足时，才产生 FIFO commit；图中 data 和 FIFO push 是因果相邻信号，不表示它们是同一根信号。DPU instruction valid 由 FIFO 当前内容组合形成，可能在 push 后立即可见，但只在下一个上升沿采样 pop。

7.2 TCU Split Response



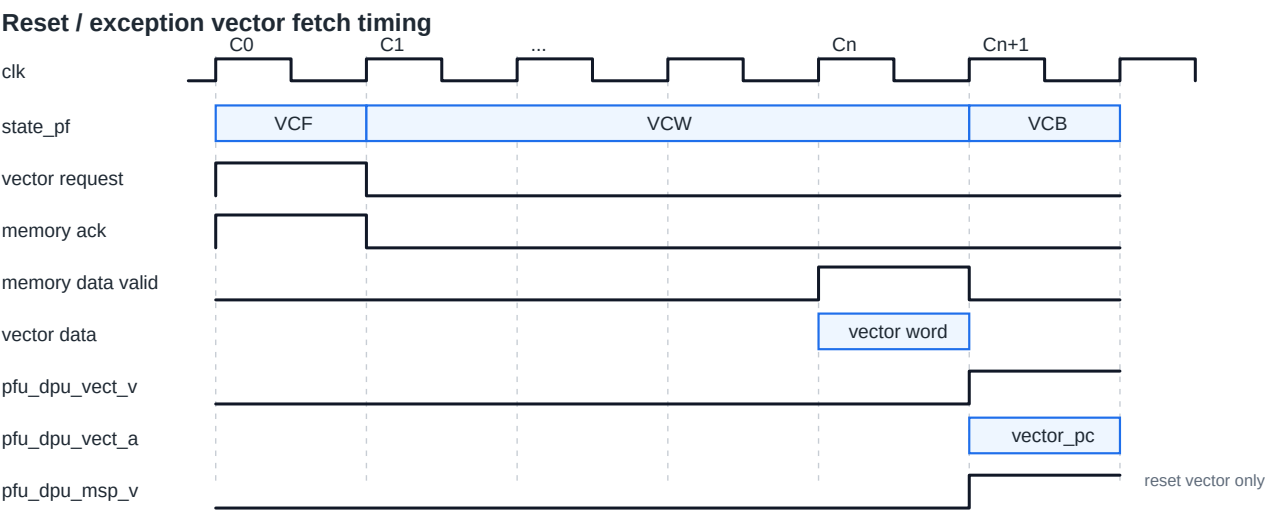
TCU 的低/高 32-bit 可由 `dvalid[0]`、`dvalid[1]` 在不同 cycle 写入输入缓冲。任何单个 word 到达都不能独立 commit；`data_complete` 表示本事务所需 64-bit 已齐备，随后 `fifo_push` 一次性建立有效窗口。若 split 中间发生 flush，已写的 word 和后续 word 均不得组成有效 fetch block。

7.3 Force During Outstanding Request



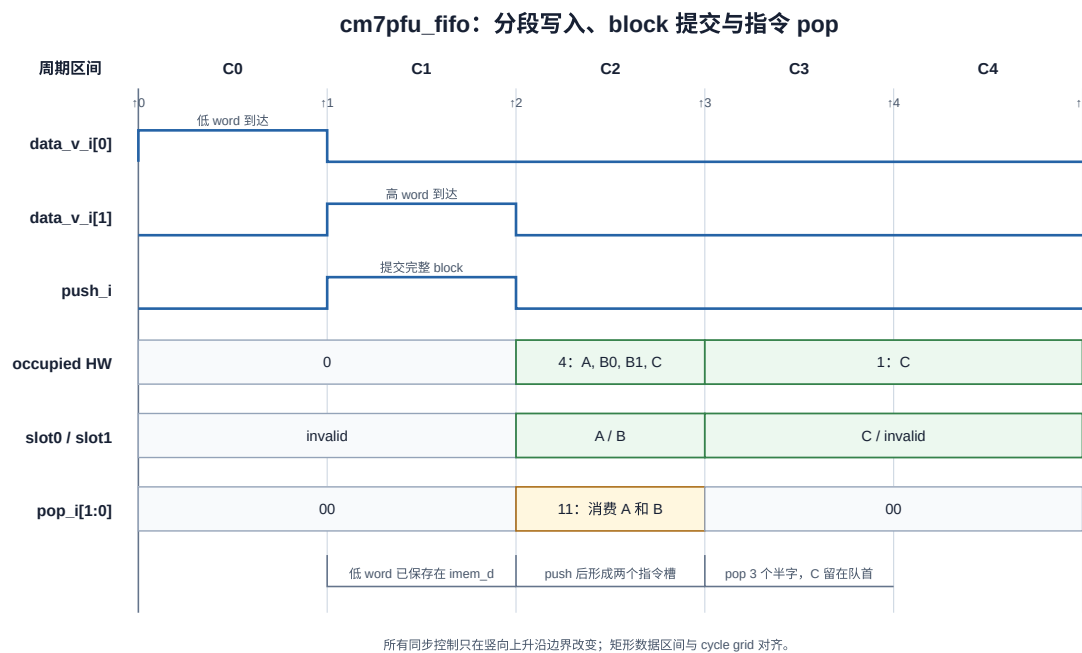
force 在任一上升沿被采样后立即覆盖下一地址并产生 flush。旧请求尚未 ack 时取消 address；已 ack 正等 data 时取消 response。旧 dvalid 即使与 force 同拍到达，也被 cancel/flush 优先屏蔽。若原 PF stall 尚不能解除，主状态进入 FRP 并保存 force address；恢复后第一笔可提交请求必须来自 force path，不能先补发旧 sequential/BTAC 地址。

7.4 Reset / Exception Vector



reset 释放且 CPU wait/MBIST 无效后，状态按 `RST -> VCF -> VCW -> VCB` 前进。VCF 发 vector table 请求，VCW 等待响应，VCB 向 DPU 同拍给出 vector PC；reset 还给出 MSP valid。memory 延迟会把 VCW 拉长，而不是改变状态顺序。vector retry/ECC error 会产生 replay 并重新进入 VCF；bus error 则产生 vector error/lockup/fault 语义。

7.5 PFQ Split Write / Push / Pop



该示例提交四个连续半字，内容依次是 16-bit 指令 A、32-bit 指令 B 的 B0/B1、16-bit 指令 C。C0 内只有 data_v_i[0]，所以低 32-bit 被保存，但 occupied HW 仍为 0，DPU 看不到 A。C1 内高 word 到达并同时给出 push_i；C2 开始有效占用变为 4，格式化结果为 slot0=A、slot1=B。

DPU 在 C2 内给出 pop_i=11，该值在 C3 起点的上升沿被采样。A 消耗一个半字，B 消耗两个半字，因此 C3 的占用变为 1，剩余 C 成为 slot0。图中所有控制变化均落在 cycle grid 上；data_v_i 表示 word 写使能，push_i 表示 block 提交，pop_i 表示 DPU 消费请求，三者不能合并成一根“FIFO 有效”信号。

8. Reimplementation Contract

8.1 不可改变与可替换边界

不可改变	可替换
cm7pfu 外部端口方向、位宽、上升沿握手、低有效 reset。	内部 signal/register 名称。
地址选择、force/interrupt/stop、错误与 cancel 优先级。	状态编码，只要外部时序和保持行为一致。
Thumb 组包、双槽输出、pop 语义、flush 清空语义。	4+12 的物理 FIFO 划分，但必须保持容量、headroom 和同周期可见行为。
BTAC 4×16 逻辑结构、entry 字段、round-robin allocation、方向转移、multiple-hit 处理。	RAM/flop 实现、比较器组织和时钟门控。
BTAC metadata 与 instruction 的顺序对齐。	side queue 的物理编码，只要容量与 backpressure 等价。

8.2 必须实现的组合优先级

1. 地址：DPU force > 状态选择的 vector/current > BTAC target > sequential。
2. 状态：interrupt/vector replay > force pending > stop > force resume > sync-error block > normal run。
3. DPU force 来源：Wr replay slot0 > Wr branch slot0 > Wr replay slot1 > Wr branch slot1 > Ex2 > Iss。
4. FE 数据：fake MPU data > ICU/TCU 根据 `cs_fe` 选择；cancel/flush 高于 commit。
5. error code：vector error/catch > first valid halfword breakpoint > MPU > bus；合法 replay 覆盖当前 FE 新错误编码但不得覆盖已存在的更早异常范围。

8.3 必须实现的断言式不变量

- `dpu_pfu_pop_iss_i[1]` -> `dpu_pfu_pop_iss_i[0]`。
- 非 flush 的 `push_i` 必须对应完整 64-bit word-valid 历史。
- `flush` 同拍 push 的数据不得在下一拍保持 valid。
- 任一有效 `first_pf=0` fetch 与最近 first fetch 的 `[31:5]` 必须相同。
- `pc_d` 和 `bt_d` 不得 overflow；full 预测必须在潜在最大写入前 stall。
- error instruction 未 pop 且仍 valid 时，error code 必须保持不变。
- BTAC `dpu_btac_inv_i[1]` 只能与 bit0 同时输入；invalidate 必须与 lookup 关联。
- 同一取指事务不得同时作为 ICU 与 TCU 的可提交响应。
- branch mispredict 后，PFU FIFO/side queues 必须为空；DPU 中所有 younger wrong-path valid 必须被 kill/quash，而 mispredicting branch 与 older instructions 不得仅因该 mispredict 被取消。

8.4 最小实现顺序

建议按以下依赖顺序重实现：先完成 PF/FE request-response 与状态机；再完成无预测的 FIFO/Thumb 组包；随后加入 vector/error/replay；最后加入 BTAC 和 side queues。每一步都应保留 flush/cancel 骨架，因为后补会改变几乎所有边界条件。

9. Verification Matrix

场景	输入激励	必查结果
Reset wait	reset 释放但 <code>cpuwait</code> 或 MBIST=1	保持 RST，无普通 request。
Reset vector	返回 MSP + reset PC	状态顺序正确；MSP/vector 同步输出；随后从 PC fetch。
ICU 可变延迟	ack 和 dvalid 分别延迟	ack 前 PF 保持；dvalid 前 FE 保持；只 commit 一次。
TCU split	两个 word 不同拍返回	complete 前无 commit；complete 后数据顺序正确。
TCM decode change	请求期间切换 enable	产生纠正/first，不混合 ICU/TCU 响应。

场景	输入激励	必查结果
FIFO 全 16-bit	连续块 + 双 pop	每拍最多两条，halfword 顺序无丢失/重复。
FIFO 全 32-bit	含跨 block 指令	第二半字未到前 invalid；完整后正确组包。
混合长度 pop	16+32、32+16、32+32	分别消费 3、3、4 halfwords。
FIFO headroom	占用跨过 8	<code>nxt_full</code> 提前节流；在途 block 不 overflow。
Flush 同拍 push/pop	三者组合	flush 后两级 buffer 均空，旧输出 invalid。
BTAC allocation wrap	每 bank 连续 17 次分配	第 17 次替换该 bank entry0。
BTAC predictor	覆盖 8 个状态/结果组合	下一状态严格匹配第 5.3 节。
BTAC multiple hit	同 bank 制造两个同 tag valid	hit 可用于维护、taken 被屏蔽、entry 自失效。
Side queue pressure	单 fetch 产生 4 hit，DPU 暂停 pop	提前 stall，无 metadata overflow/错位。
Correct taken prediction	target 与实际一致	无 flush；target 指令连续进入 DPU。
Direction mispredict	taken/not-taken 两方向	wrong path 全部清除，从 force 地址恢复。
Target mispredict	taken 但 offset 错	force 到正确 target，并 full-update BTAC。
Selective branch recovery	在 branch 前放置有写回的 older instruction，在 branch 后放置寄存器写、store 和 slot1 指令	older instruction 与 branch 正常完成；所有 younger wrong-path register/PC/LSU side effect 被屏蔽；PFU FIFO 清空。
Force during ack wait	request 未 ack 时 force	address cancel；FRP 保存 force。
Force with data same cycle	dvalid 与 force 同拍	旧 data 不 commit。
Stop during stall	FE 等待时 stop	<code>blp_v</code> 记忆，解除后进 BLK。
MPU abort	P1 abort	fake data 推动 code <code>0010</code> ，真实响应取消。
FPB + protection	breakpoint hit，分别设置 protection	未保护产生 <code>0001</code> ；保护时屏蔽。

场景	输入激励	必查结果
Bus error	ICU/TCU error	对应 halfword error 与 code 0011 对齐。
Replay	已 push 后 retry/ECC	code 0101；普通/vector 路径分别重取。
Vector faults	reset、NMI/HardFault、普通异常 bus error，并分别组合 vector catch	低 3 bit 分别为 000/110/111，bit3 与 catch 正交；vector error 输出正确。

10. Known Unknowns

- PF stage 与 FE stage 的英文全称未在当前可读 RTL 中给出；本文只规定其功能，不猜测名称展开。这不阻塞实现。
- hash_addr 内部计算了 addr_inv，但返回值明确是 addr[14:3]。兼容本 RTL 必须使用实际返回值；addr_inv 是否为历史遗留或其他配置用途不影响当前实现。
- RAR 改变部分数据寄存器的 reset 值，但不改变 valid/state 的功能语义。若目标要求所有数据寄存器可预测复位，需要按 RAR=1 验证。
- 本文覆盖功能 RTL，不规定 clock gating、scan、Design For Test (DFT，面向测试设计)、功耗意图和物理时序约束。